

VLSI Design and Testing Lab (BECL606)

List Of Experiments

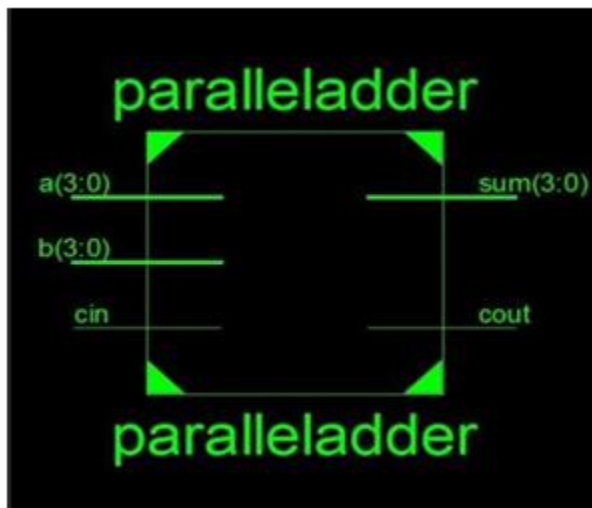
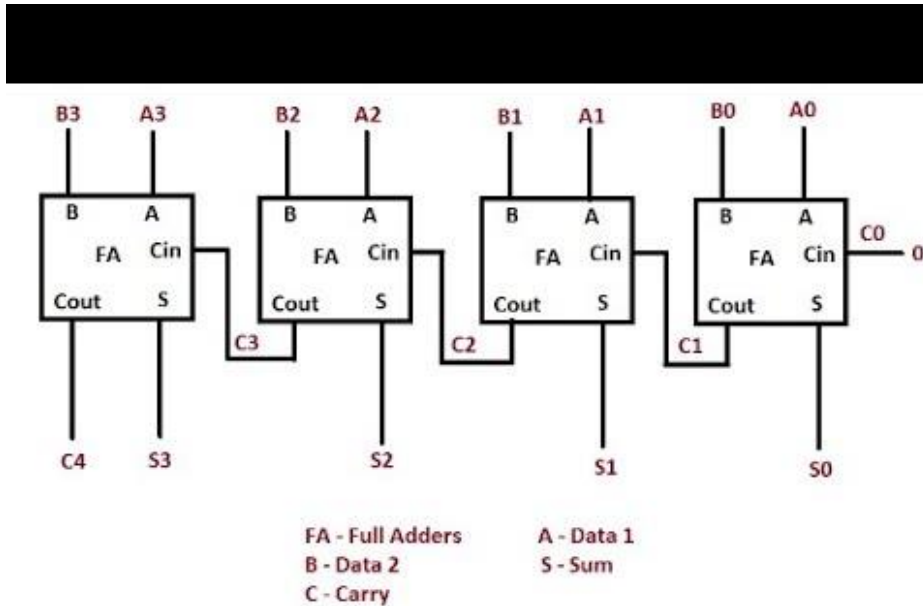
1. Design a 4-Bit Adder
2. 4-Bit Shift and add Multiplier
3. 32-Bit ALU Supporting 4-Logical and 4-Arithmetic operations, using case and if statement for ALU Behavioral Modeling
4. Flip-Flops (D,SR and JK)
5. Four bit Synchronous MOD-N counter with Asynchronous reset
6. Construct the schematic of CMOS inverter and Draw layout of inverter with $W_p/W_n = 40/20$
7. Capture the schematic of 2-input CMOS NOR gate
8. Construct the schematic of the Boolean Expression $Y = AB + CD + E$ using CMOS Logic.
9. Construct the schematic of Common Source Amplifier with PMOS Current Mirror Load and find its transient response and AC response
10. Construct the schematic of two-stage operational amplifier
11. UART
12. Design and characterize 6T binary SRAM cell

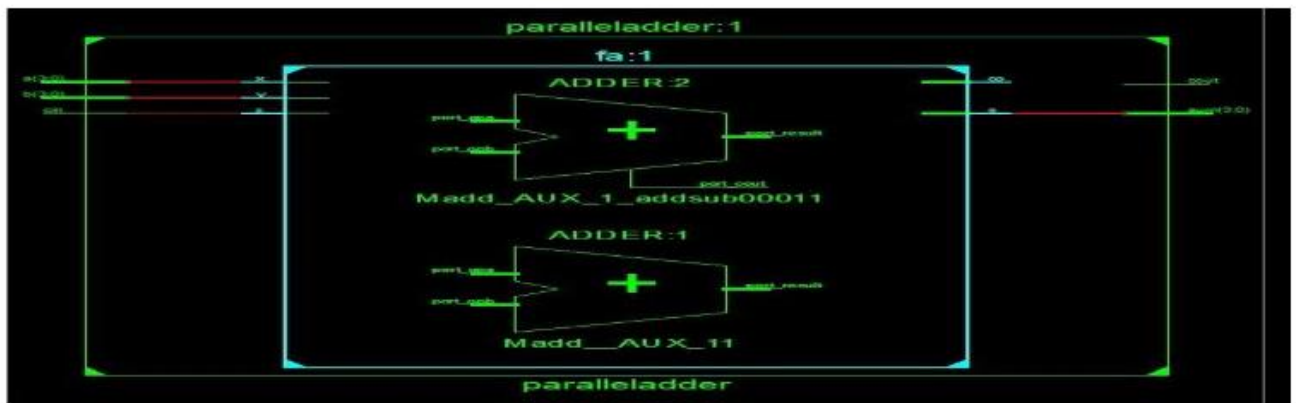
VIVA Questions

1.A 4-Bit Adder

AIM: To write Verilog code for 4-bit adder and its test bench for verification and observe the waveform and synthesize the code.

BLOCK DIAGRAM



RTL SCHEMATIC DIAGRAM:**PROGRAM:**

```
module parallelladder(a,b,cin,sum,cout); //Declares a Verilog module named
```

```
parallelladder
```

```
input[3:0]a;
```

```
input[3:0]b; input cin;
```

```
output[3:0]sum;
```

```
output cout;
```

```
wire[2:0]cint; fa fa1(a[0],b[0],cin,sum[0],cint[0]);
```

```
fa fa2(a[1],b[1],cint[0],sum[1],cint[1]);
```

```
fa fa3(a[2],b[2],cint[1],sum[2],cint[2]);
```

```
fa fa4(a[3],b[3],cint[2],sum[3],cout);
```

```
endmodule
```

```
module fa(x,y,z,s,co);
```

```
input x,y,z;
```

```
output s,co;
```

```
assign {co,s}=x+y+z;
```

```
endmodule
```

TESTBENCH:

```
module paralleladder_tb;
```

```
reg[3:0]a;
```

```
reg[3:0]b
```

```
reg cin;
```

```
wire[3:0]sum;
```

```
wire cout;
```

```
paralleladder fbpa1(a,b,cin,sum,cout);
```

```
initial
```

```
begin
```

```
a=4'b0000;
```

```
b=4'b1001;
```

```
cin=1'b0;
```

```
#200;
```

```
a=4'b0110;
```

```
b=4'b1001;
```

```
cin=1'b0;
```

```
#200;
```

```
a=4'b0100;
```

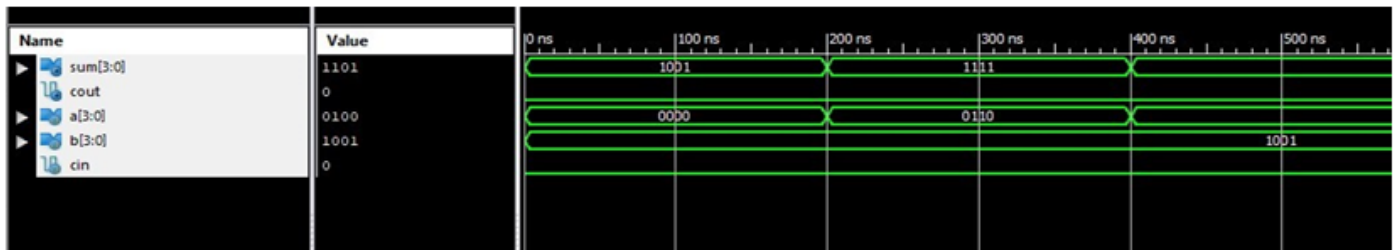
```
b=4'b1001;
```

```
cin=1'b0;
```

```
#200;
```

```
end
```

```
endmodule
```

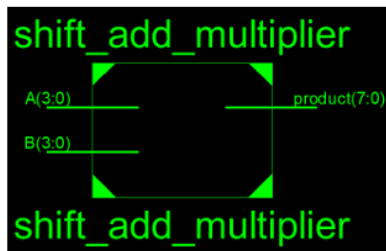
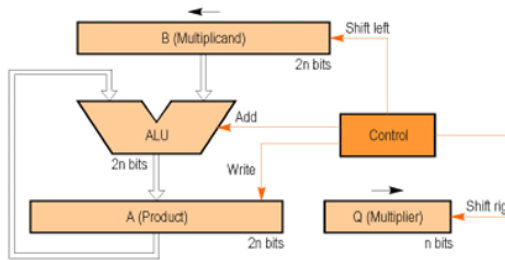
WAVEFORM:

RESULT: Verilog code for 4-bit adder and its test-bench for verification are written, the waveform is observed and the code is synthesized.

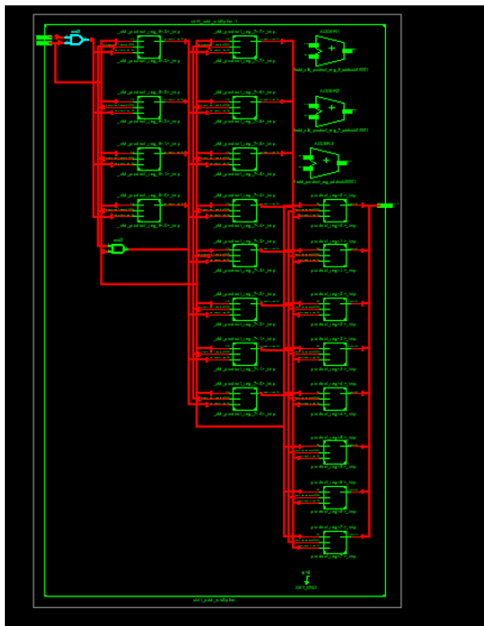
2. 4-Bit Shift and add Multiplier

AIM: To write Verilog code for 4-Bit Shift and add Multiplier and its test bench for verification and observe the waveform and synthesize the code.

BLOCK DIAGRAM:



RTL SCHEMATIC DIAGRAM:



PROGRAM:

```
module shift_add_multiplier (  
    input [3:0] A,  
    input [3:0] B,  
    output [7:0] product // 8-bit product ); // The final multiplication result (since  
    multiplying two 4-bit numbers can produce an 8-bit result).  
    reg [7:0] product_reg; // Stores the intermediate and final product result.  
    reg [7:0] A_reg; // Stores an extended version of A (with 4 extra bits for  
    shifting).  
    reg [7:0] B_reg; // Stores an extended version of B (same as A_reg).  
    integer i; // Loop variable used for iteration in the multiplication process.  
    // Perform multiplication using shift and add  
    always @(A, B) // executes whenever A or B changes.  
    begin // Implements the shift-and-add multiplication algorithm.  
        A_reg = {4'b0000, A}; // Extend A to 8 bits (padding with zeros)  
        B_reg = {4'b0000, B}; // Extend B to 8 bits (padding with zeros)  
        product_reg = 8'b00000000; // Loop for 4 times for shift and add  
        for (i = 0; i < 4; i = i + 1) // The loop runs 4 times since we are multiplying  
            two 4-bit numbers.  
            begin if (B_reg[i]) begin // Checks each bit of B reg (multiplier).  
                product_reg = product_reg + A_reg;  
            end  
            A_reg = A_reg << 1; // The multiplicand (A_reg) is shifted left by 1 bit at  
            each iteration.  
        end  
    end
```

```
end  
  
assign product = product_reg; // The computed product is assigned to the  
output product  
  
endmodule
```

TESTBENCH:

```
module test_shift_add_multiplier; // Declares the testbench module  
  
// Testbench signals  
  
reg [3:0] A; // Stores the multiplicand for testing  
reg [3:0] B; // Stores the multiplier for testing  
wire [7:0] product; // Captures the multiplication result from the DUT (Design  
Under Test).  
  
// Instantiate the shift-and-add multiplier  
shift_add_multiplier uut ( .A(A), .B(B), .product(product) );  
  
initial begin // start of the simulation.  
  
// Test Case 1: 3 * 2  
  
A = 4'b0011; // A = 3  
B = 4'b0010; // B = 2  
  
#10;  
  
$display("Test Case 1: A = %b, B = %b, product = %b", A, B, product);  
  
// Test Case 2: 4 * 5  
  
A = 4'b0100; // A = 4  
B = 4'b0101; // B = 5  
  
#10; // Waits 10 time units for the multiplication to complete  
  
$display("Test Case 2: A = %b, B = %b, product = %b", A, B, product);
```



```
// Prints the inputs and the computed product in binary format.

// Test Case 3: 8 * 3

A = 4'b1000; // A = 8

B = 4'b0011; // B = 3

#10;

$display("Test Case 3: A = %b, B = %b, product = %b", A, B, product);

// Test Case 4: 7 * 6

A = 4'b0111; // A = 7

B = 4'b0110; // B = 6

#10;

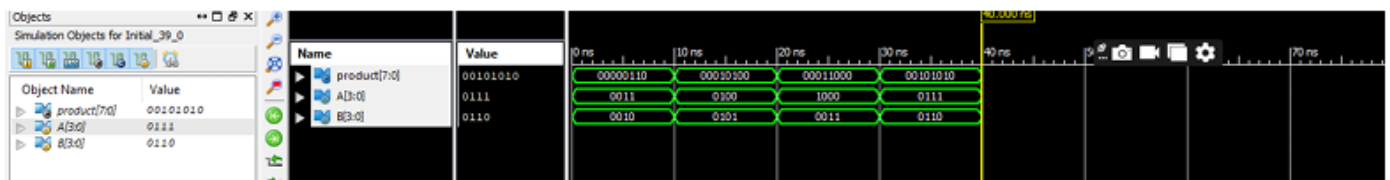
$display("Test Case 4: A = %b, B = %b, product = %b", A, B, product);

$finish; //terminates the simulation after all test cases are executed

end

endmodule
```

WAVEFORM:

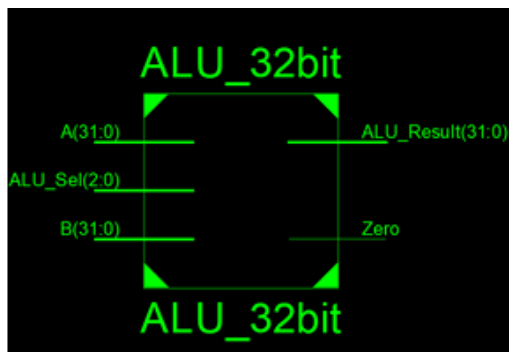


RESULT: Verilog code for 4-Bit Shift and add Multiplier and its test-bench for verification are written, the waveform is observed and the code is synthesized.

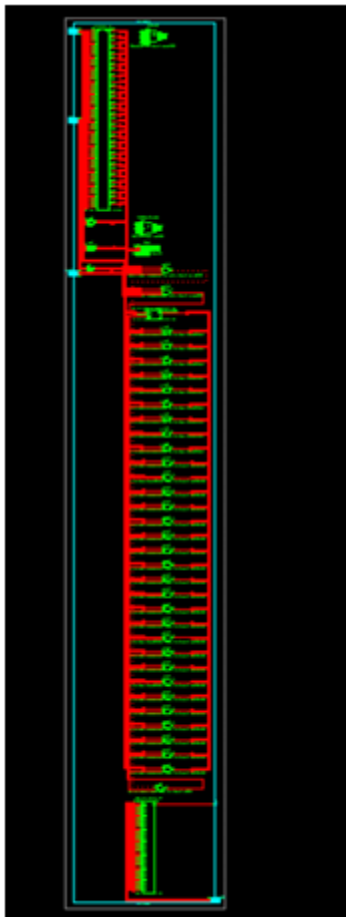
3.32-Bit ALU

AIM: To write Verilog code for 32-bit ALU and its test bench for verification and observe the waveform and synthesize the code.

BLOCK DIAGRAM:



RTL SCHEMATIC DIAGRAM:



PROGRAM:

//Module Declaration

module ALU_32bit (

input [31:0] A, B, //32-bit input operands

input [2:0] ALU_Sel, // 3-bit selection signal that determines which

operation the ALU performs.

output reg [31:0] ALU_Result, //A 32-bit register that holds the result of the
selected operation.

output reg Zero //A flag that is set when the result is zero.

);

always @(*) begin //it executes whenever any of its input

case (ALU_Sel) //Case Statement for ALU Operations

3'b000: ALU_Result = A + B; // Addition If ALU_Sel is 000,

perform addition: A + B

3'b001: ALU_Result = A - B; // Subtraction

3'b010: ALU_Result = A * B; // Multiplication

3'b011: ALU_Result = A + 1 ; // increment

3'b100: ALU_Result = A & B; // AND

3'b101: ALU_Result = A | B; // OR

3'b110: ALU_Result = A ^ B; // XOR

3'b111: ALU_Result = ~A; // NOT (on A)

default: ALU_Result = 32'b0;

endcase

if (ALU_Result == 32'b0)

Zero = 1; //If ALU_Result is zero, set Zero = 1 (indicating a zero

```
result).  
    else  
        Zero = 0; //Otherwise, set Zero = 0  
end  
endmodule //Marks the end of the module definition.
```

TESTBENCH:

```
module test; Defines the testbench module  
  
    // Inputs  
    reg [31:0] A; // 32-bit register to give input A to ALU  
    reg [31:0] B; //32-bit registers to hold the input values.  
    reg [2:0] sel; //3-bit register used to select the ALU operation.  
  
    // Outputs  
    wire [31:0] Y;  
  
    // Instantiate the Unit Under Test (UUT)  
    alu uut (.A(A),.B(B), .sel(sel), .Y(Y));  
  
    initial begin  
        // Initialize Inputs  
        A = 32'b00000111;  
        B = 32'b00001111;  
  
        // Wait 100 ns for global reset to finish  
        #100; sel=3'b000;  
        #100; sel=3'b001;  
        #100; sel=3'b010;  
        #100; sel=3'b011;
```

```

#100; sel=3'b100;

#100; sel=3'b101;

#100; sel=3'b110;

#100; sel=3'b111;

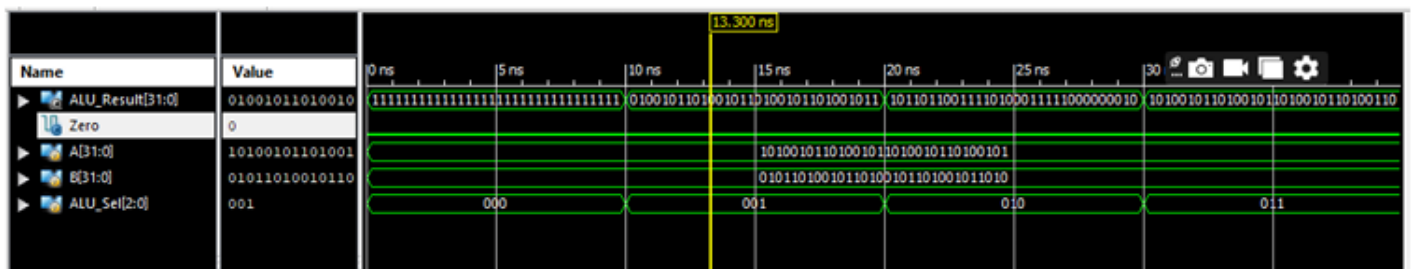
// Add stimulus here

end

endmodule

```

Wave form:

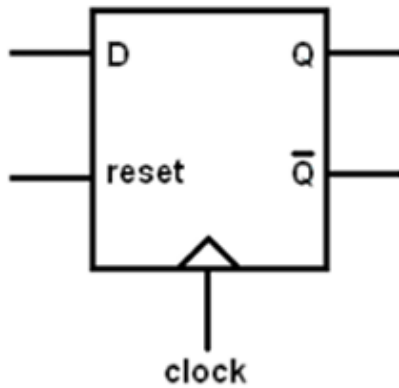


Result: Verilog code for 32-bit ALU and its test-bench for verification are written, the waveform is observed and the code is synthesized.

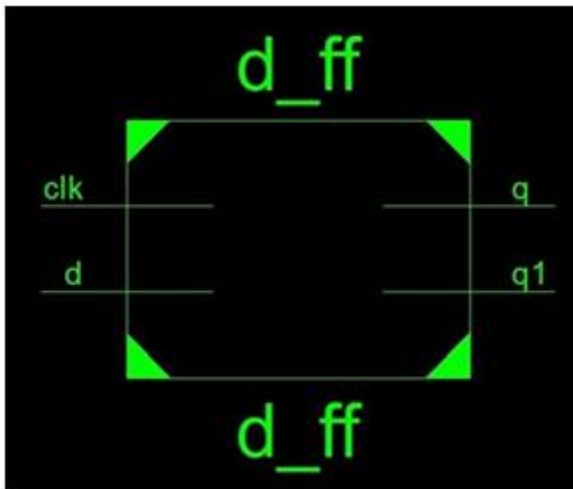
4.Flip-Flop

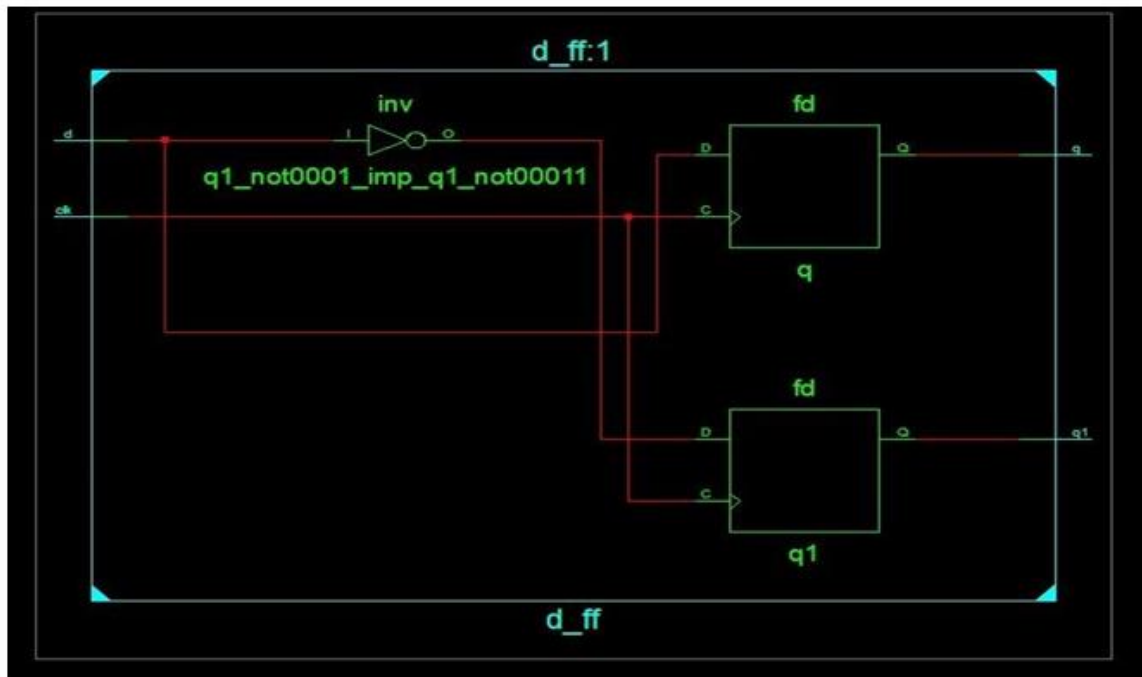
AIM: To write Verilog code for flip-flop and its test bench for verification and observe the waveform and synthesize the code.

BLOCK DIAGRAM:



Input			Output	
D	reset	clock	Q	Q'
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	1	0
1	1	0	0	1
1	1	1	0	1



RTL SCHEMATIC DIAGRAM:**D-Flip-flop****Program:**

```

module d_ff(d,clk,q,q1); // Module definition
input d,clk; // the data input and clock input.
output q,q1; // the outputs
reg q,q1; //reg because they will hold values between clock cycles.
initial begin
q=1'b0;q1=1'b1; // q is initially set to 0. // q1 is initially set to 1.
end
always @(posedge clk) //The block will execute when the clock signal
transitions from 0 to 1.
begin

```

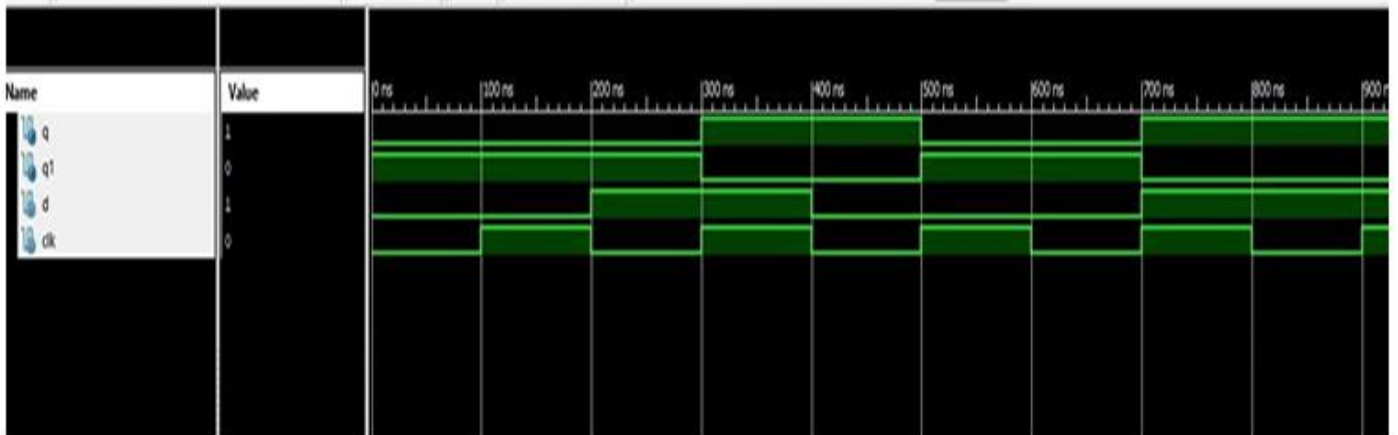
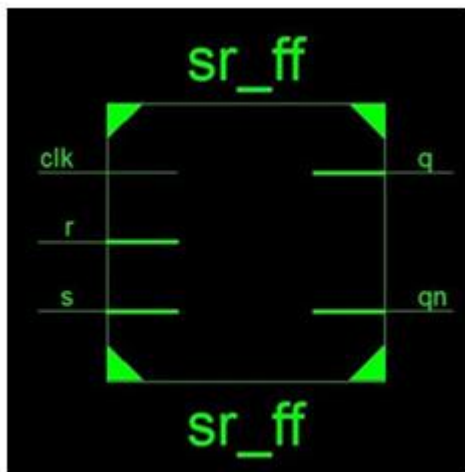
```
q=d; //q will be updated to match the value of d
q1=~d; // The output q1 will be the inverse of d
end
endmodule
```

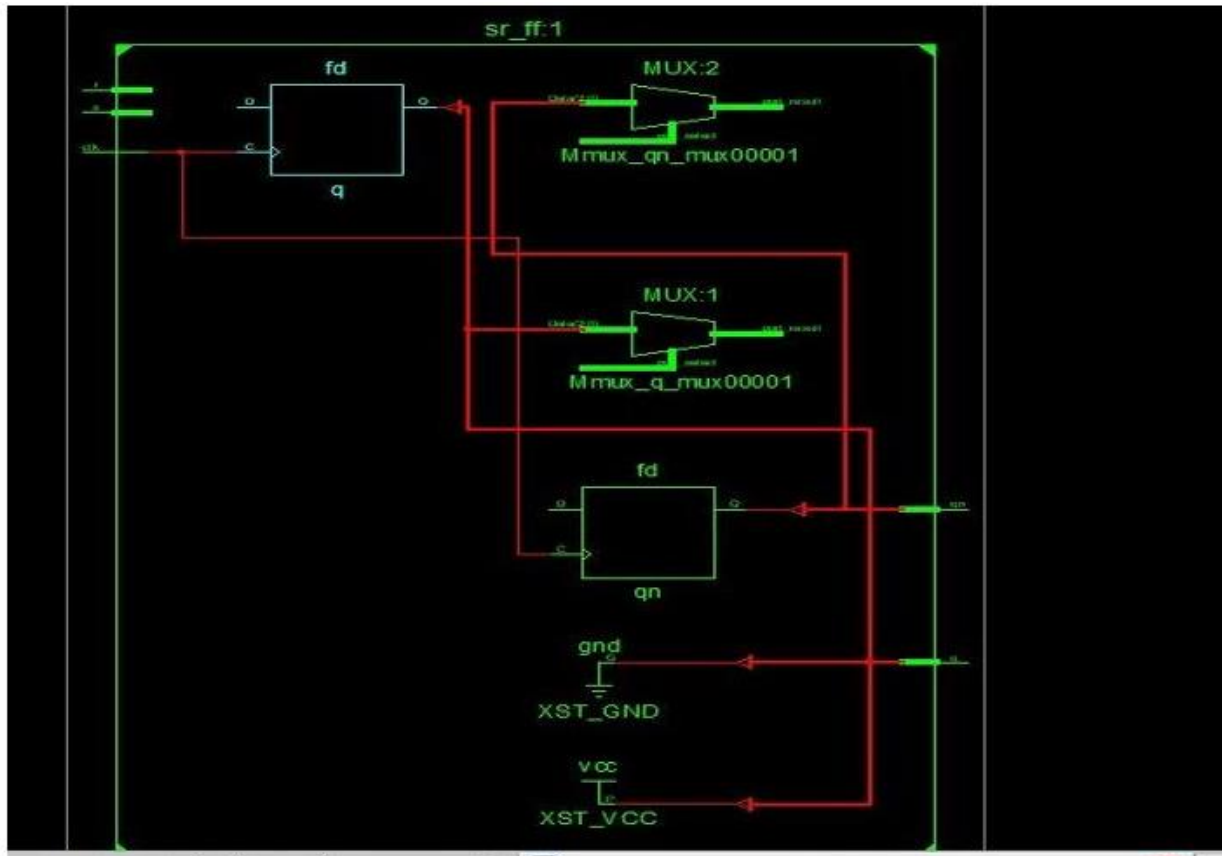
TESTBENCH:

```
module test; // Declaration Module
reg d,clk; // The data input and clock signal for the D flip-flop
wire q,q1; // The outputs of the D flip-flop
d_ff uut(.d(d),.clk(clk),.q(q),.q1(q1)); // the instantiates the d_ff module within
the testbench
initial
clk = 1'b0; // This block initializes the clock signal clk to 0 at the
start of the simulation
always
#100 clk = ~clk; // the clock signal (clk) will alternate between 0 and
1 every 100 time units
initial begin
d=1'b0;#200; // The signal d is set to 0 for 200 time units.
d=1'b1;#200; // The signal d is set to 1 for 200 time units.
d=1'b0;#300; // d is set to 0 for 300 time units.
d=1'b1;#400; // d is set to 1 for 400 time units.
end
initial
```



```
#2000 $finish; // This initial block ends the simulation after 2000 time unit  
Endmodule
```

WAVEFORM:**SR- Flip-flop****BLOCK DIAGRAM**

RTL SCHEMATIC DIAGRAM:**SR- Flip-flop****PROGRAM:**

```

module srff(s,r,en,q,qn); // Declaration of Module
input s,r,en; // Set, reset and enable input
output q,qn; // the outputs
reg q,qn; // Declares q and qn as reg types because their values are assigned in an always block.
initial begin // initial values of q and qn
q=1'b0; //q is initialized to 0.
qn=1'b1; // qn is initialized to 1
end

```

```

always @(posedge en) // This is triggered on the positive edge of the enable
signal
begin
case({s,r}) // case statement that checks the values of the Set and Reset inputs
{1'b0,1'b0}:begin q=q;qn=qn;end
{1'b0,1'b1}:begin q=1'b0;qn=1'b1;end
{1'b1,1'b1}:begin q=1'b1;qn=1'b0;end
{1'b1,1'b1}:begin q=1'bx;qn=1'bx;end
endcase
end
endmodule //ending the module

```

TESTBENCH:

```

module test; // defines the test module
reg s,r,clk; // inputs to the SR Flip-Flop
wire q,qn; // output of the SR Flip-Flop
srff uut(.s(s),.r(r),.clk(clk),.q(q),.qn(qn));
// This instantiates the srff module
and connects the testbench signals to the SR Flip-Flop's inputs and outputs.
initial
// Clock Generation
clk=1'b1; // sets the clk signal to 1
always #100 clk=~clk; // This always block toggles the clk signal every 100
time units
initial begin

```

```
// Test Stimuli for s and r

s=0;r=0;#200;

s=0;r=1;#200;

s=1;r=0;#200;

s=1;r=1;#200;

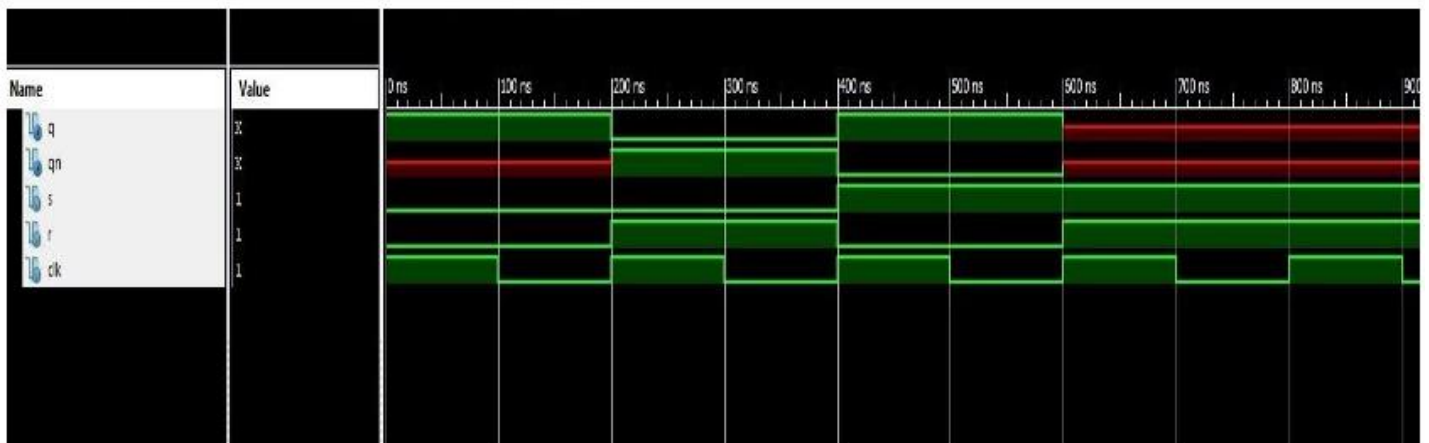
end

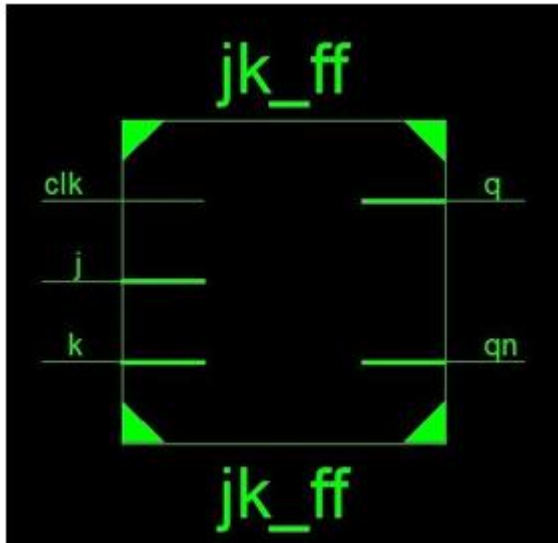
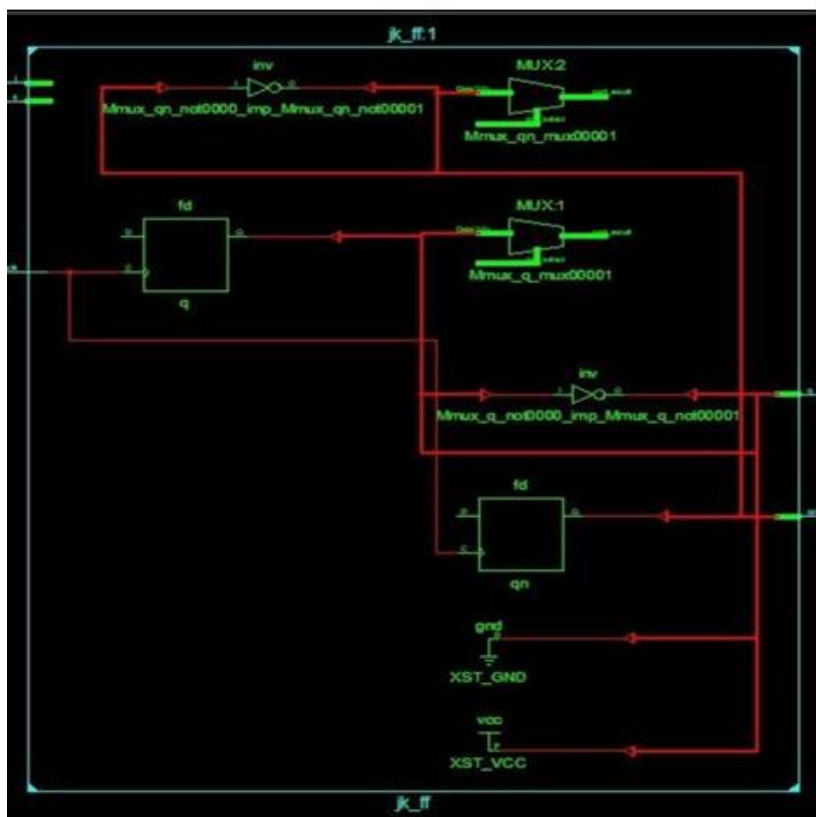
initial

#2000 $finish; // This initial block ensures that the simulation finishes after

2000 time units

Endmodule
```

WAVEFORM:

JK- Flip-flop**BLOCK DIAGRAM:****RTL SCHEMATIC DIAGRAM:**

PROGRAM:

```
module jkff(j,k,clk,q,qn); //defines the module jkff with inputs and outputs
input j,k,clk; // input signals
output q,qn; // output signals
reg q,qn; //q and qn will store values, they are declared as register type
initial begin
q=1'b0; // It initializes q to 0.
qn=1'b1; // It initializes qn to 1
end
always @(posedge clk) // execute on the rising edge of the clock
begin
case({j,k}) // Case Statement (Defining JK Flip-Flop Behavior)
{1'b0,1'b0}:begin q=q;qn=qn;end
// Case Conditions
{1'b0,1'b1}:begin q=1'b0;qn=1'b1;end
{1'b1,1'b0}:begin q=1'b1;qn=1'b0;end
{1'b1,1'b1}:begin q=~q;qn=~qn;end
endcase
end
endmodule //module ends
```

TESTBENCH:

```
module test; //defines the test module

reg j,k,clk; //Registers store values for inputs

wire q,qn; //Wires are used to receive output signals

jkff uut(.j(j),.k(k),.clk(clk),.q(q),.qn(qn)); //Instantiating the JK Flip-Flop (Unit
Under Test)

initial //Initializing Clock Signal

clk=1'b0; //Sets the initial clock value to 0.

always #100clk=~clk; //Generating Clock Signal (Toggling Every 100 Time
Units)

initial //Applying Test Inputs

begin

j=0;k=0;#200; // Apply J=0, K=0 for 200 time units (No change)

j=1;k=0;#400; //Apply J=1, K=0 for 400 time units (Set Q=1)

j=0;k=1;#600; //Apply J=0, K=1 for 600 time units (Reset Q=0)

j=1;k=1;#800; //Apply J=1, K=1 for 800 time units (Toggle Q)

end

initial

#2000 $finish; //The simulation stops after 2000 time units

Endmodule
```

WAVEFORM:

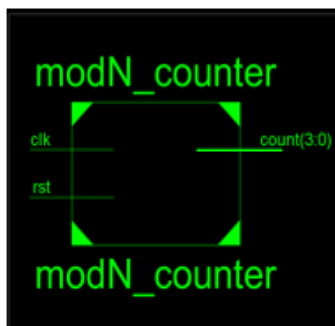


RESULT: Verilog code for flip-flop and its test-bench for verification are written, the waveform is observed and the code is synthesized.

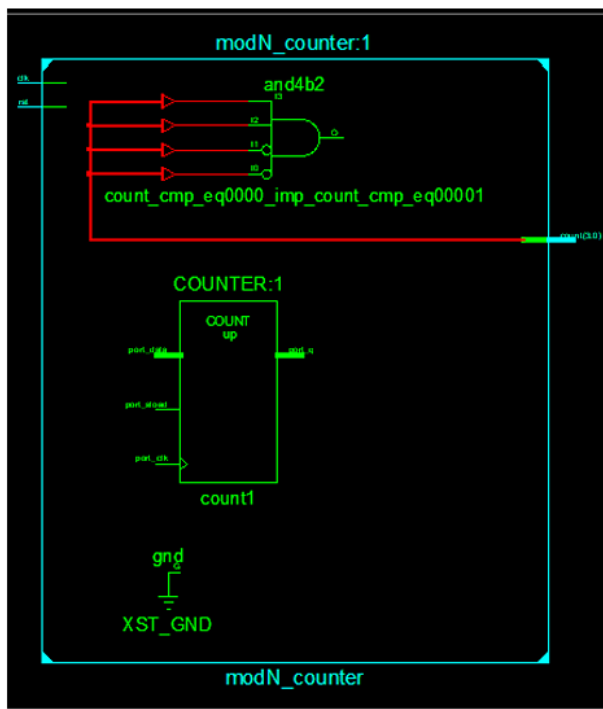
5.Four-Bit Synchronous Mod-N Counter with Asynchronous Reset

AIM: To write Verilog code for Four-bit Synchronous MOD-N counter with Asynchronous reset and its test bench for verification and observe the waveform and synthesize the code.

BLOCK DIAGRAM:



RTL SCHEMATIC DIAGRAM:



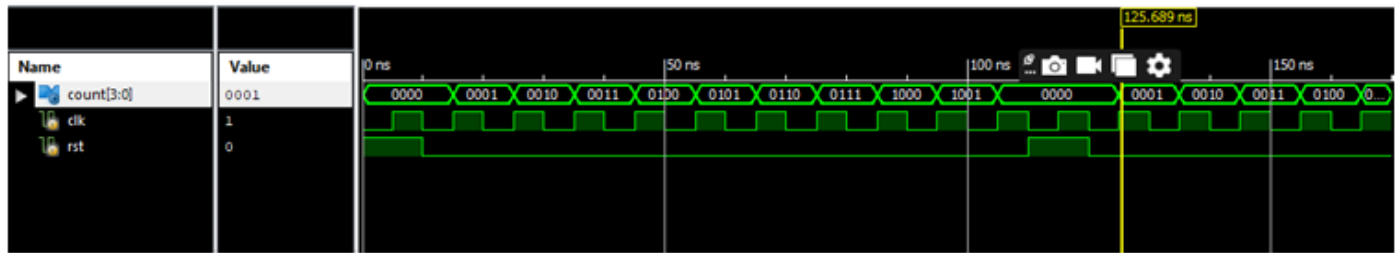
PROGRAM:

```
module modN_counter #(parameter N = 10) (  
    input clk, rst, // Declares the input signals clk (clock) and rst (reset)  
    clk is used for counting, and rst is used for asynchronous reset.  
    output reg [3:0] count); // Declares a 4-bit output register  
    always @(posedge clk or posedge rst) begin // this means the block  
        will execute whenever there is a clock edge (for counting) or a reset signal (to  
        reset the counter).  
        if (rst) // rst signal is high  
            count <= 0; // Asynchronously resets the count to 0  
        else if (count == N - 1) //  
            count <= 0; // the counter will be reset to 0 in the next clock  
        cycle.  
        else  
            count <= count + 1; // counter is incremented.  
        end  
    endmodule
```

TESTBENCH

```
module modN_counter_tb; // Defines the testbench module  
    reg clk, rst; // Declares clk and rst as registers  
    wire [3:0] count; // Declares count as a 4-bit wire that will hold the output of  
    the counter.  
  
    // Instantiate counter with MOD-10
```

```
modN_counter #(.N(10)) uut (  
    .clk(clk),  
    .rst(rst),  
    .count(count)  
);  
  
// Generate Clock (10 ns period)  
always #5 clk = ~clk; // This generates a clock signal with a period of 10 ns.  
Every 5 ns, the value of clk is toggled  
  
initial begin  
    $dumpfile("modN_counter_tb.vcd"); // This opens a file  
modN_counter_tb.vcd where waveform data will be written  
    $dumpvars(0, modN_counter_tb); // This tells the simulator to dump all  
signals in the testbench to the .vcd file.  
  
    clk = 0; rst = 1; // Initial state  
    #10 rst = 0;      // Release reset  
    #100;             // Run simulation  
    rst = 1;          // Apply reset  
    #10 rst = 0;      // Release reset  
    #50;              // Run for some more cycles  
    $stop;  
end  
endmodule
```

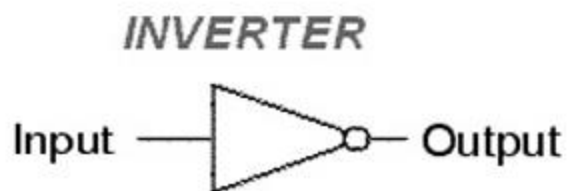
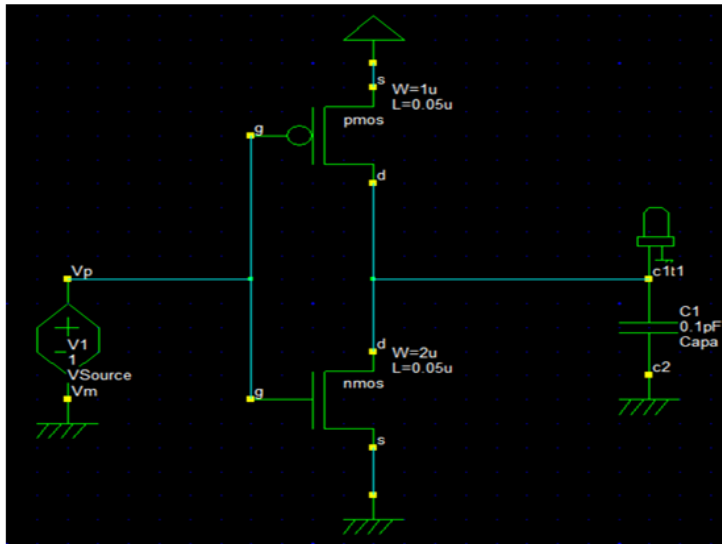
WAVEFORM:

RESULT: Verilog code for Four-bit Synchronous MOD-N counter with Asynchronous reset and its test-bench are written and verified, the waveform is observed and the code is synthesized.

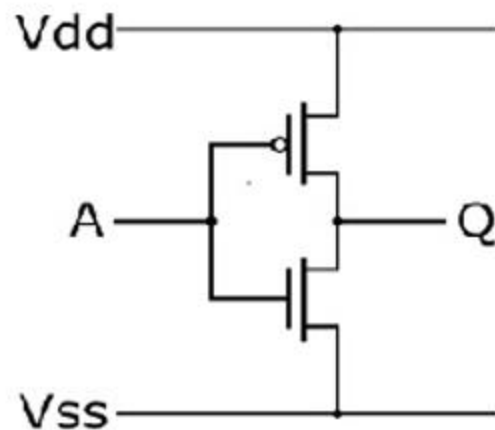
6.INVERTER

AIM: To simulate the schematic of the CMOS inverter, and then to perform the physical verification for the layout of a CMOS inverter.

SCHEMATIC DIAGRAM:

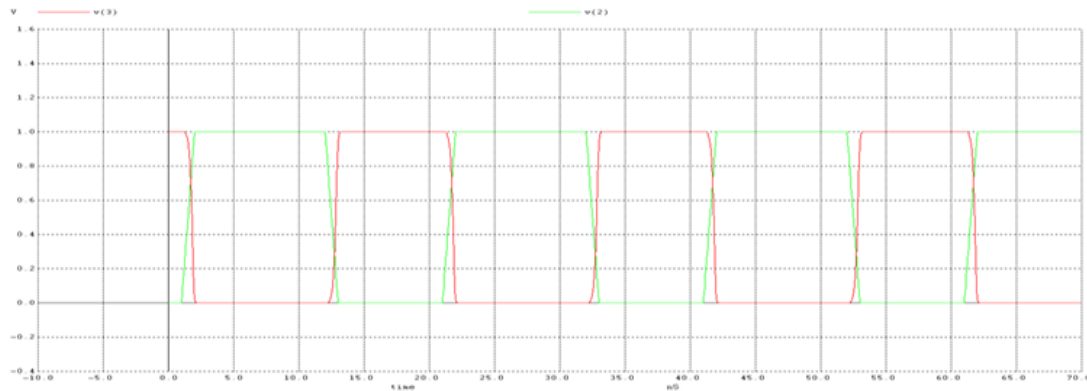
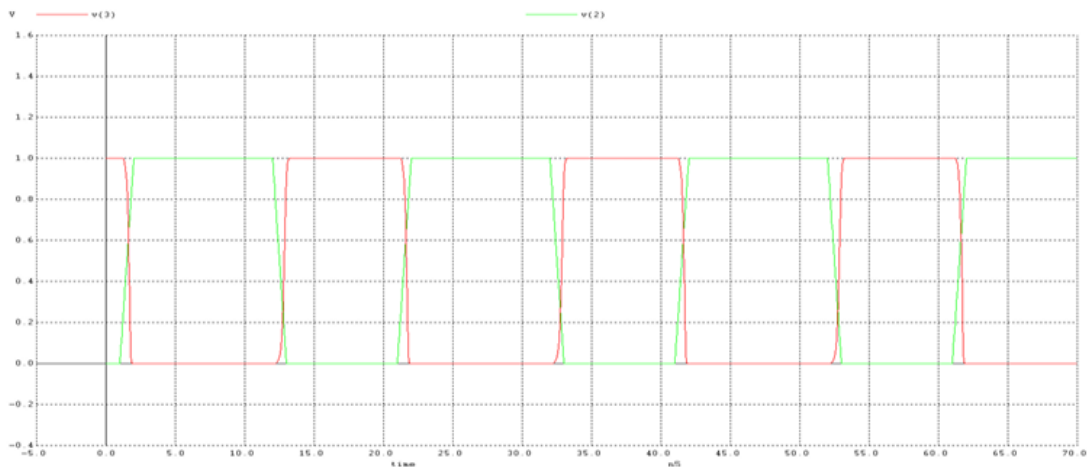


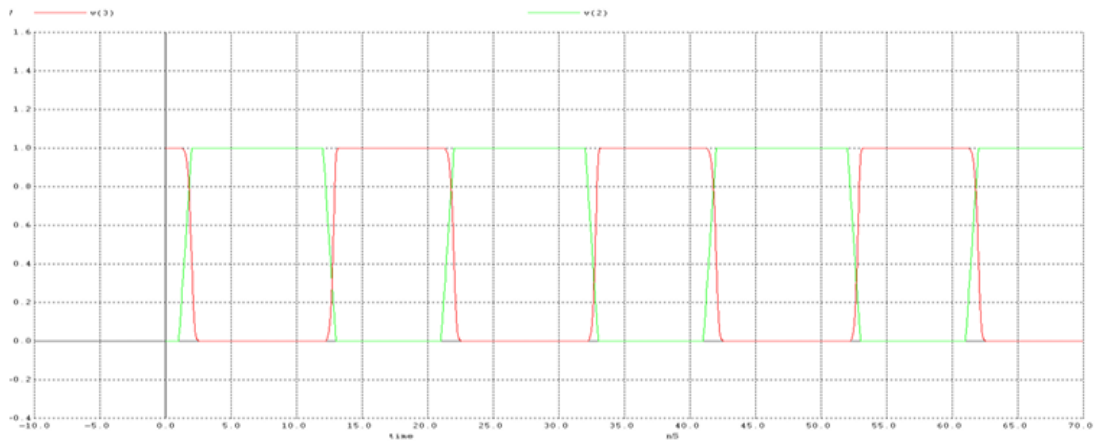
Input	Output
1	0
0	1



SPECIFICATIONS:**PMOS:** $W=1\mu$, $L=0.05\mu$ **NMOS:** Case i) $W_n=W_p$; $W=1\mu$, $L=0.05\mu$ Case ii) $W_n=2W_p$; $W=2\mu$, $L=0.05\mu$ Case iii) $W_n=W_p/2$; $W=0.5\mu$, $L=0.05\mu$ **DC Parameters:**

Values (V): 1.0

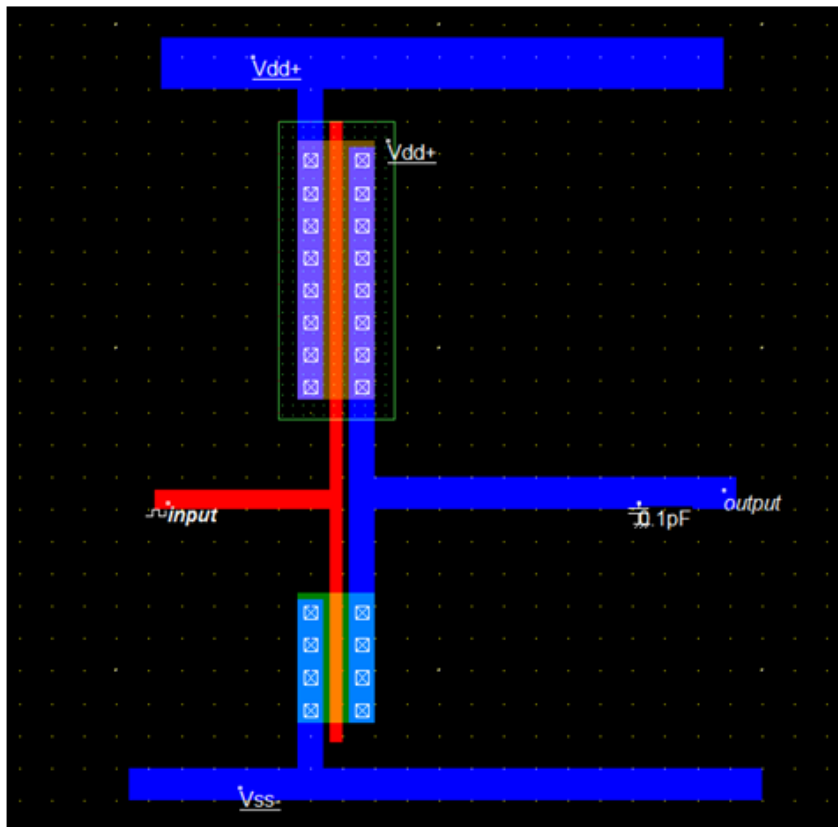
Pulse parameters: T_D (ns) = 1, T_R (ns) = 1, T_F (ns) = 1 P_W (ns) = 10, Period (ns) = 20**Load Capacitance:** 0.1pF**Case (i):** $W_n=W_p$;**Case (ii):** $W_n=2W_p$;**Case (iii):** $W_n=4W_p$;

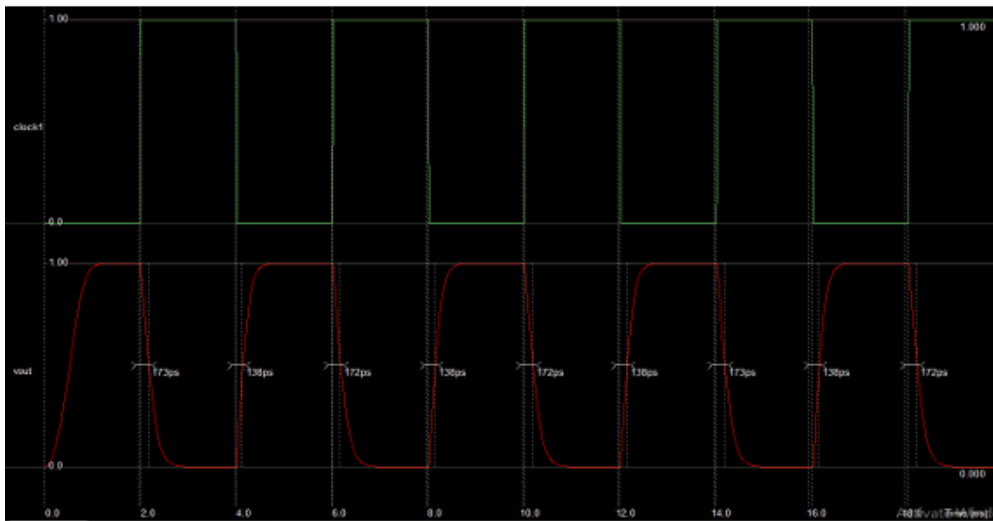
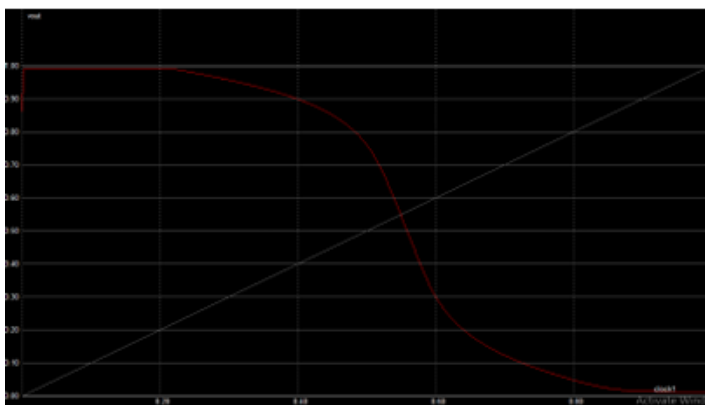


LAYOUT:

PMOS: W=800nm, L=40nm

NMOS: W=400nm, L=40nm



LAYOUT OUTPUT WAVEFORM:**VOLTAGE VERSUS VOLTAGE OUTPUT WAVEFORM:****SPECIFICATIONS:**

Clock: time low=2 , rise time=0.010, time high=2, fall time=0.010

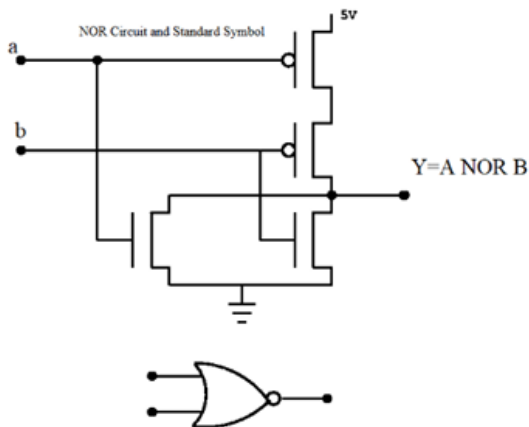
Load Capacitance: 0.1pF

RESULTS: The schematic and layout for the inverter is drawn and verified.

7.CMOS NOR GATE

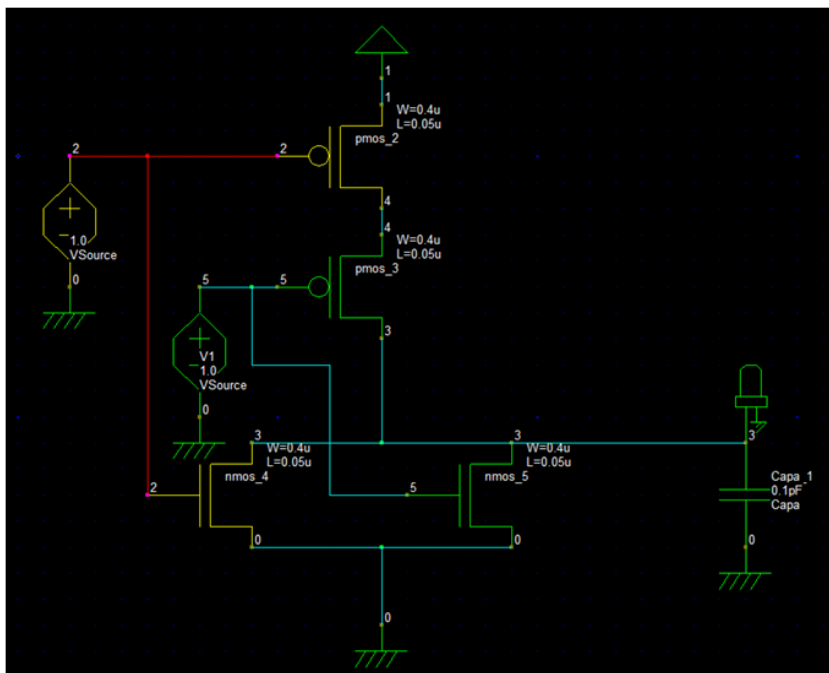
AIM: To simulate the schematic of the CMOS 2 input NOR gate, and then to perform the physical verification for the layout of an the CMOS 2 input NOR gate.

CMOS NAND GATE:



V_{G1}	V_{G2}	T_1	T_2	V_{out}
0	0	<i>off</i>	<i>off</i>	1
0	1	<i>off</i>	<i>on</i>	0
1	0	<i>on</i>	<i>off</i>	0
1	1	<i>on</i>	<i>on</i>	0

SCHEMATIC DIAGRAM:



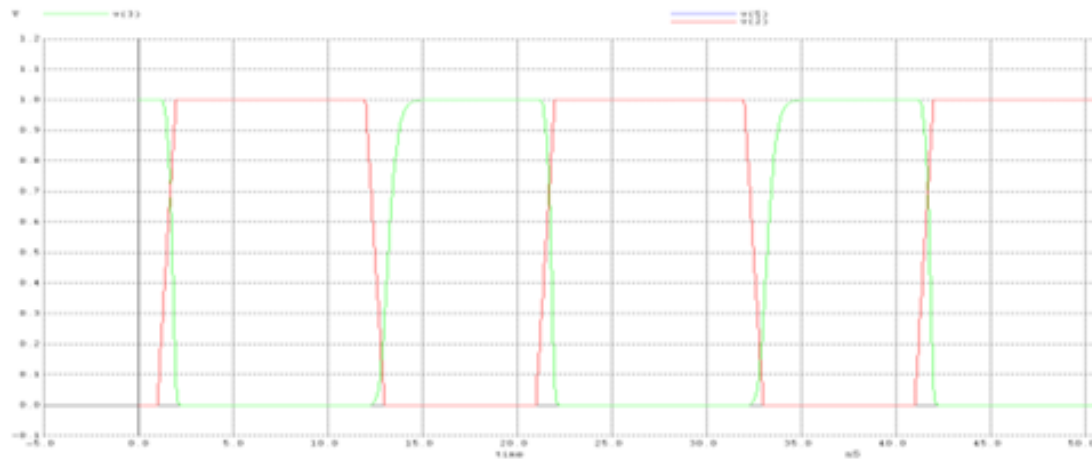
SPECIFICATIONS:**PMOS:** $W_p=0.4\mu$, $L=0.05\mu$ **NMOS:** Case i) $W_n=W_p$; $W_n=0.4\mu$, $L=0.05\mu$ Case ii) $W_n=2W_p$; $W_n=0.8\mu$, $L=0.05\mu$ Case iii) $W_n=4W_p$; $W_n=1.6\mu$, $L=0.05\mu$ **DC Parameters:**

Values (V): 1.0

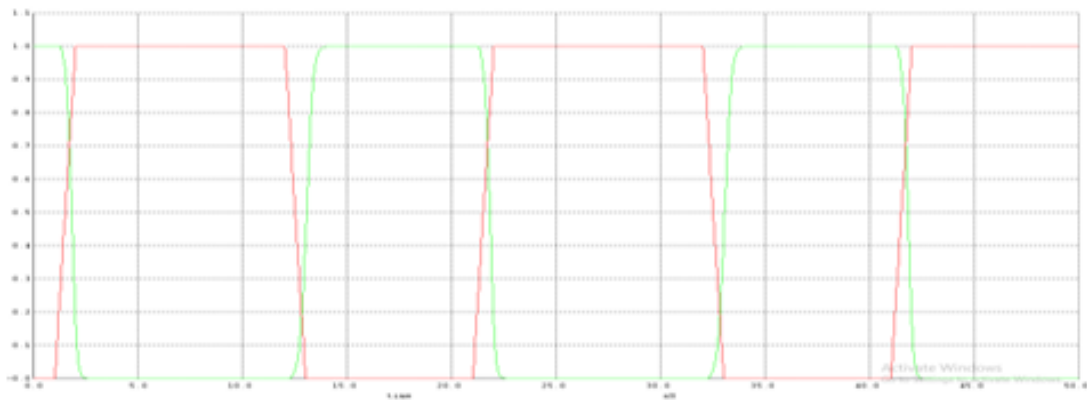
Pulse parameters: T_D (ns) = 1, T_R (ns) = 1, T_F (ns) = 1 P_W (ns) = 10, Period (ns) = 20**Load Capacitance:** 0.1pF**CALCULATIONS:****Case (i):** $W_n=W_p$; **0.4 μ** $t_{PLH}=\dots\dots\dots$, $t_{PHL}=\dots\dots\dots$ $T_D = (t_{PLH} + t_{PHL}) / 2 = \dots\dots\dots$ **Case (ii):** $W_n=2W_p$; **0.8 μ** $t_{PLH}=\dots\dots\dots$, $t_{PHL}=\dots\dots\dots$ $T_D = (t_{PLH} + t_{PHL}) / 2 = \dots\dots\dots$ **Case (iii):** $W_n=4W_p$; **1.6 μ** $t_{PLH}=\dots\dots\dots$, $t_{PHL}=\dots\dots\dots$ $T_D = (t_{PLH} + t_{PHL}) / 2 = \dots\dots\dots$

Width	T_{PLH}	T_{PHL}	T_D

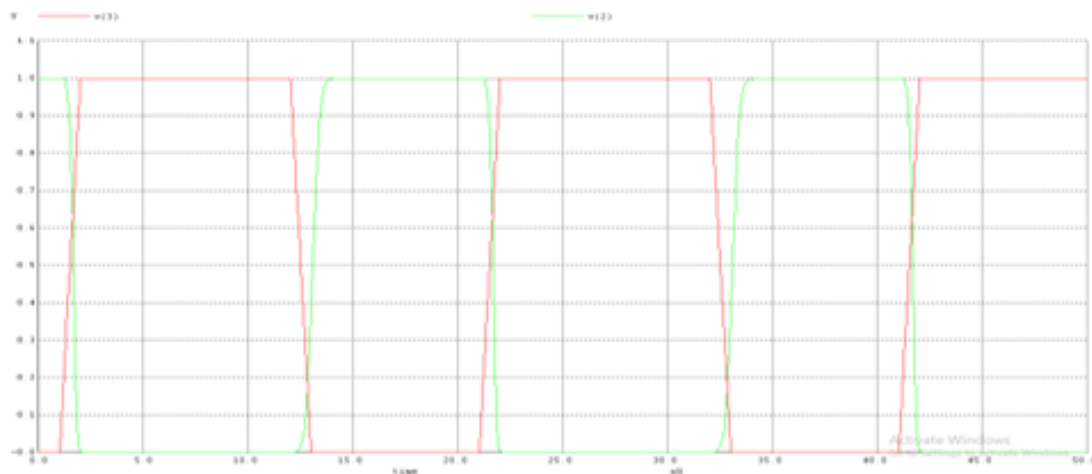
Case (i): $W_n = W_p$;



Case (ii): $W_n = 2W_p$;



Case (iii): $W_n = 4W_p$;

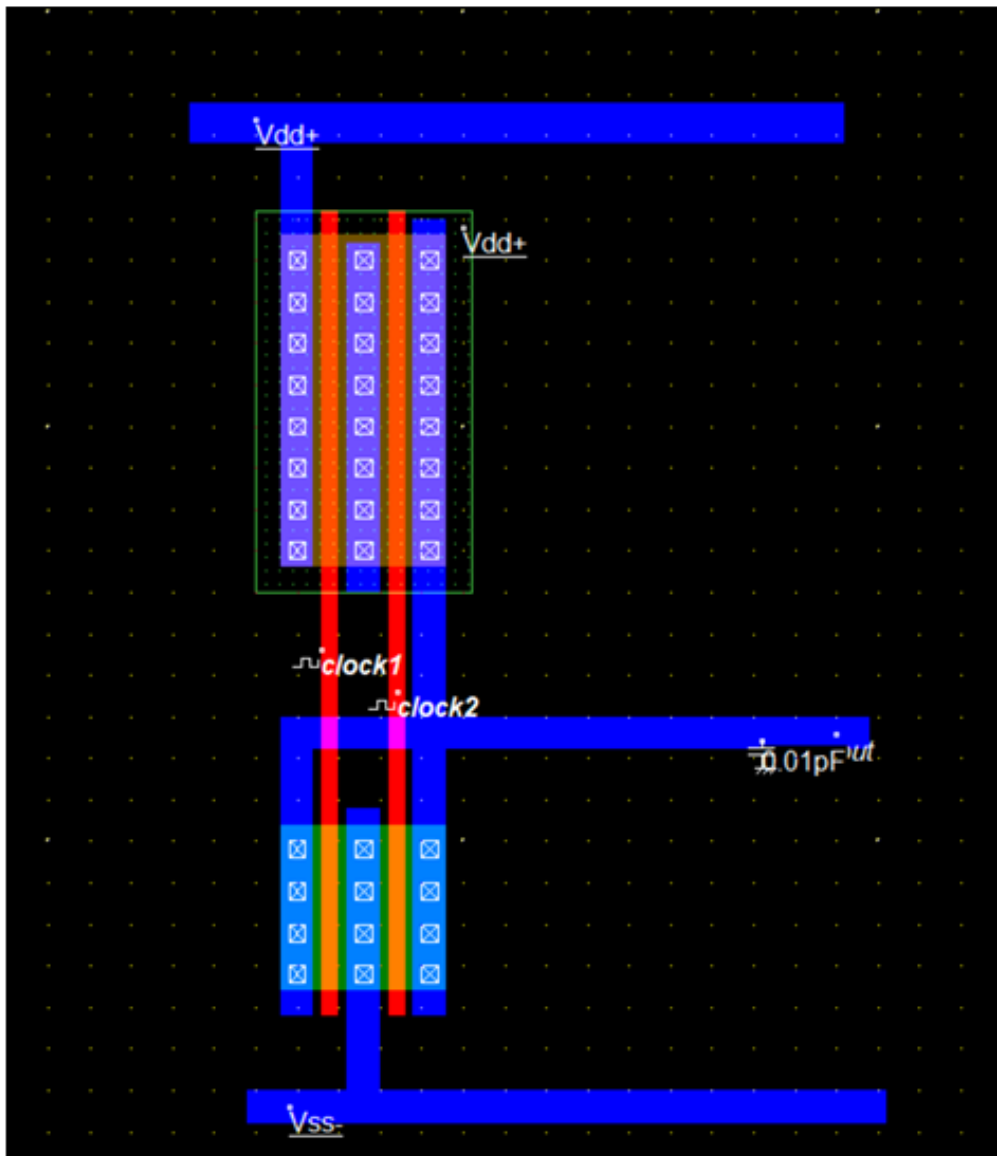


LAYOUT:

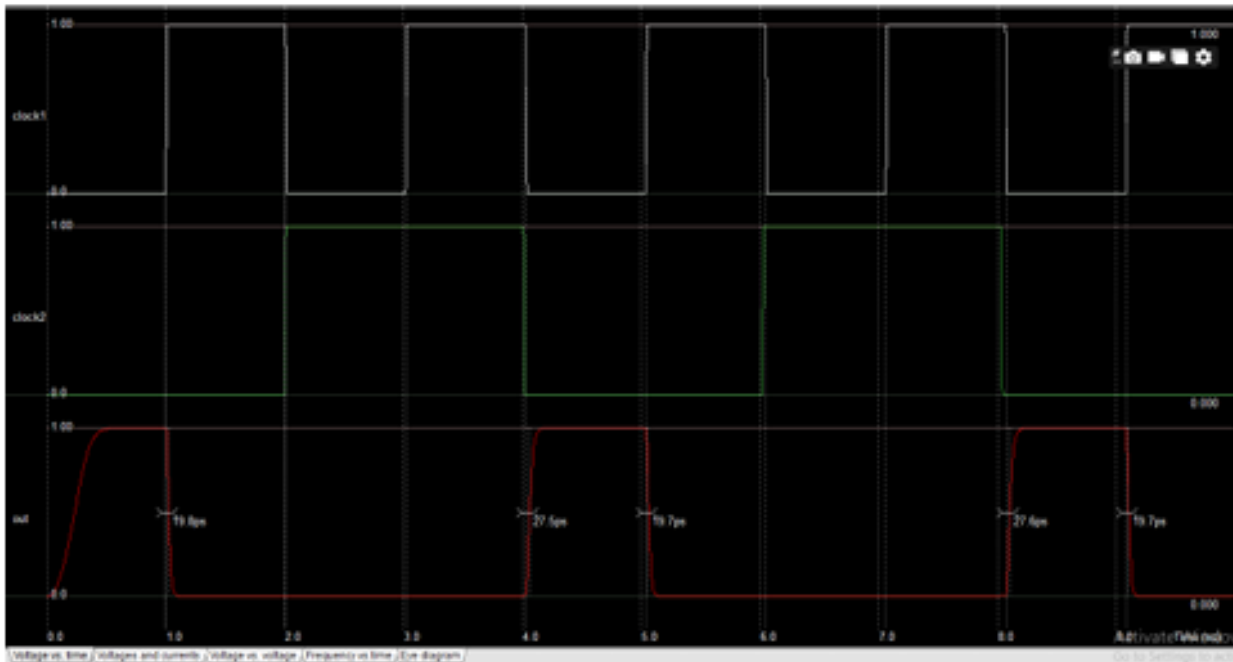
PMOS: W=800nm, L=40nm

NMOS: W=400nm, L=40nm

Number of Fingers: 2



Layout output waveform:

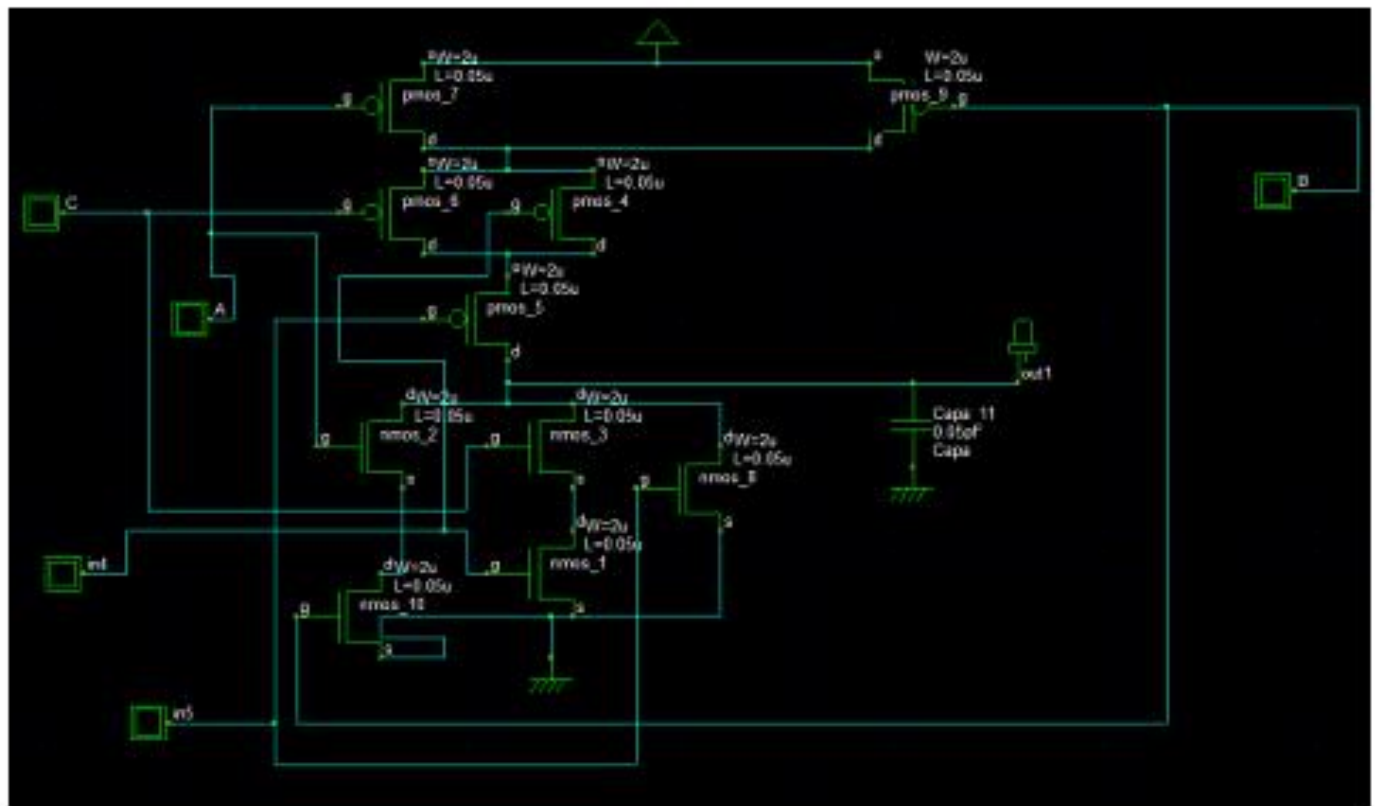


RESULTS: The schematic and layout for the CMOS 2-input NOR gate is drawn and verified.

8. Construct the schematic using Boolean Expression using CMOS-Logic ($Y = AB+CD+E$)'

AIM: To simplify and analyse the Boolean function ($Y = AB + CD + E$) and verify the functionality of the expression.

SCHEMATIC DIAGRAM:



SPECIFICATIONS:

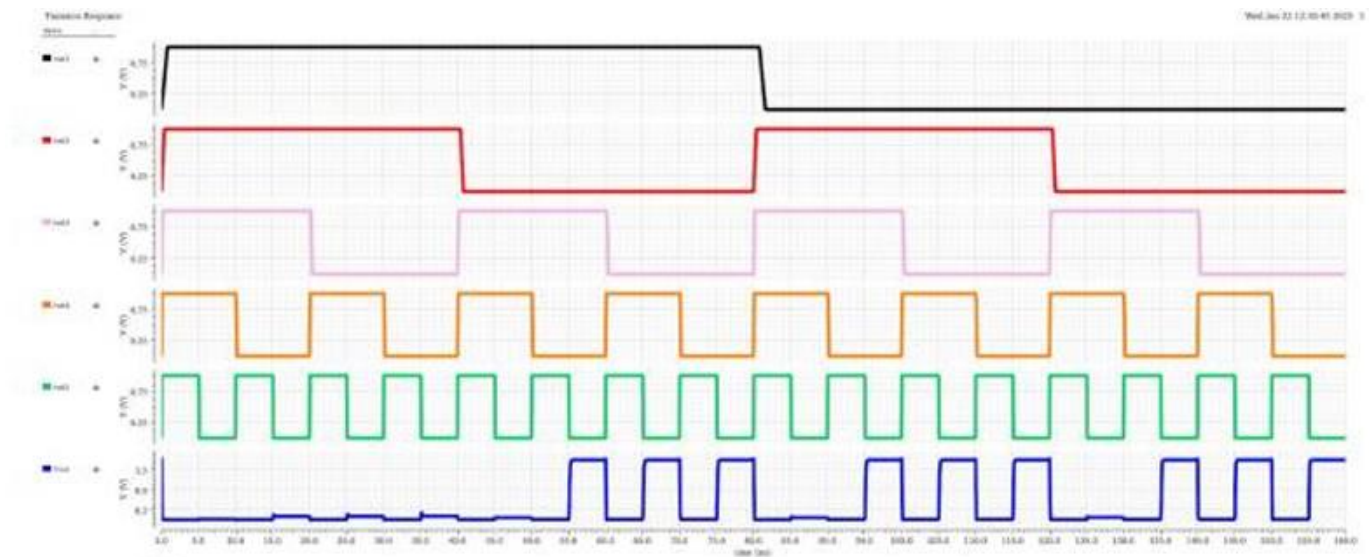
i) Voltage source: Simple

DC Parameters:

Values (V): 0.6

OBSERVATIONS:

A	B	C	D	E	Y (Expected)
0	0	0	0	0	0
1	1	0	0	0	1 (AB = 1)
0	0	1	1	0	1 (CD = 1)
0	0	0	0	1	1 (E = 1)
1	1	1	1	1	1

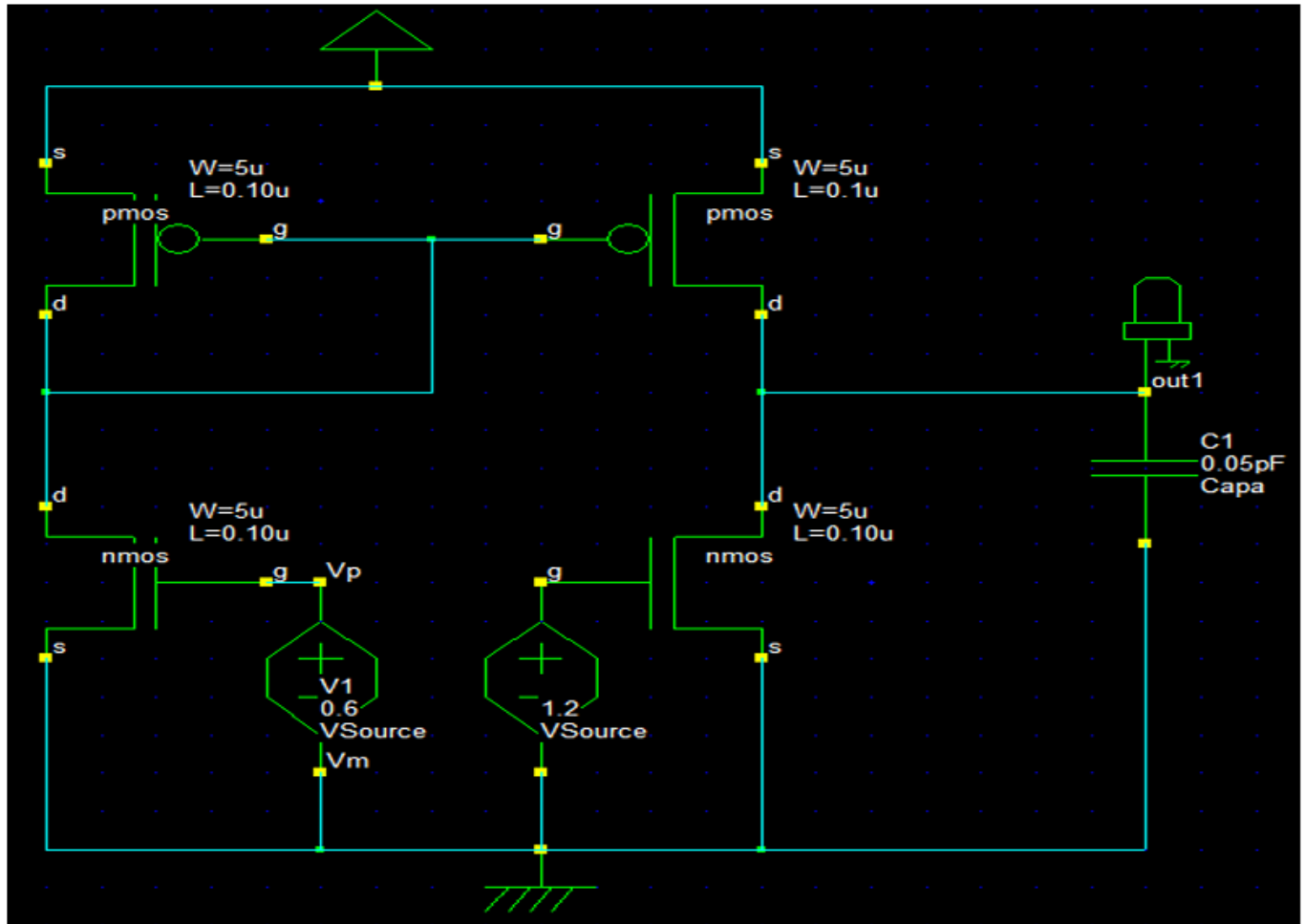
WAVEFORMS:

RESULTS: The schematic for Boolean expression is drawn and verified.

9.COMMON SOURCE AMPLIFIER

AIM: To simulate the schematic and to draw the layout of the common source amplifier.

SCHEMATIC DIAGRAM:



SPECIFICATIONS:

PMOS: $W_p=5.0\mu$, $L=0.10\mu$

NMOS: $W_n=5.0\mu$, $L=0.10\mu$

i) Voltage source: Simple

DC Parameters:

Values (V): 0.6

ii) Voltage source:

DC Parameters:

Values (V): 1.2

Sinus parameters: V_O (V) = 0.6 V_a (V) = 0.1

T_D (ns) = 0.0 Θ = 0.0

Case i) frequency (MHz) = 500

Case ii) frequency (MHz) = 4000

Case iii) frequency (MHz) = 10000

Load Capacitance: 0.05 pF

Output waveform Observation

i) Transient Response:

Analysis: Transient

Frequency/Time:

From 1ns to 100ns

Plot: Linear

ii) AC Response:

Analysis: AC

Frequency/Time:

From 200MHz to 10GHz

Plot: dB

Gain Calculation

i) Transient Analysis:

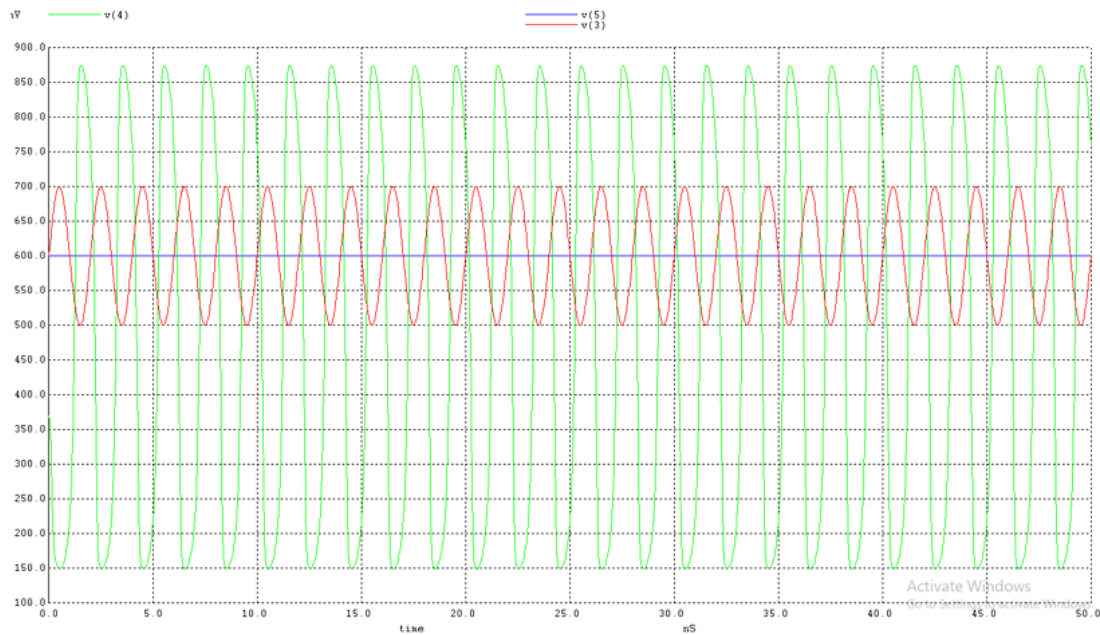
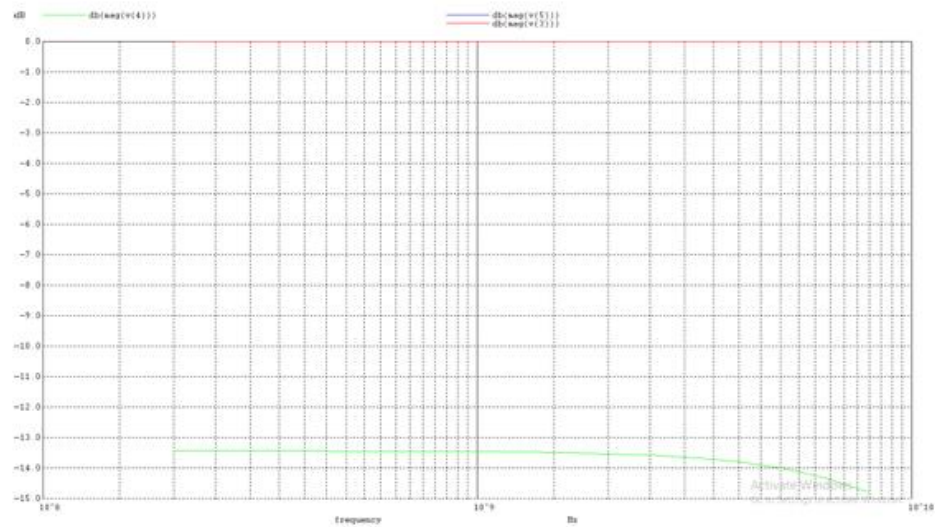
Input:

Output:

Gain: $\frac{\text{Output}}{\text{Input}} =$

ii) AC Analysis:

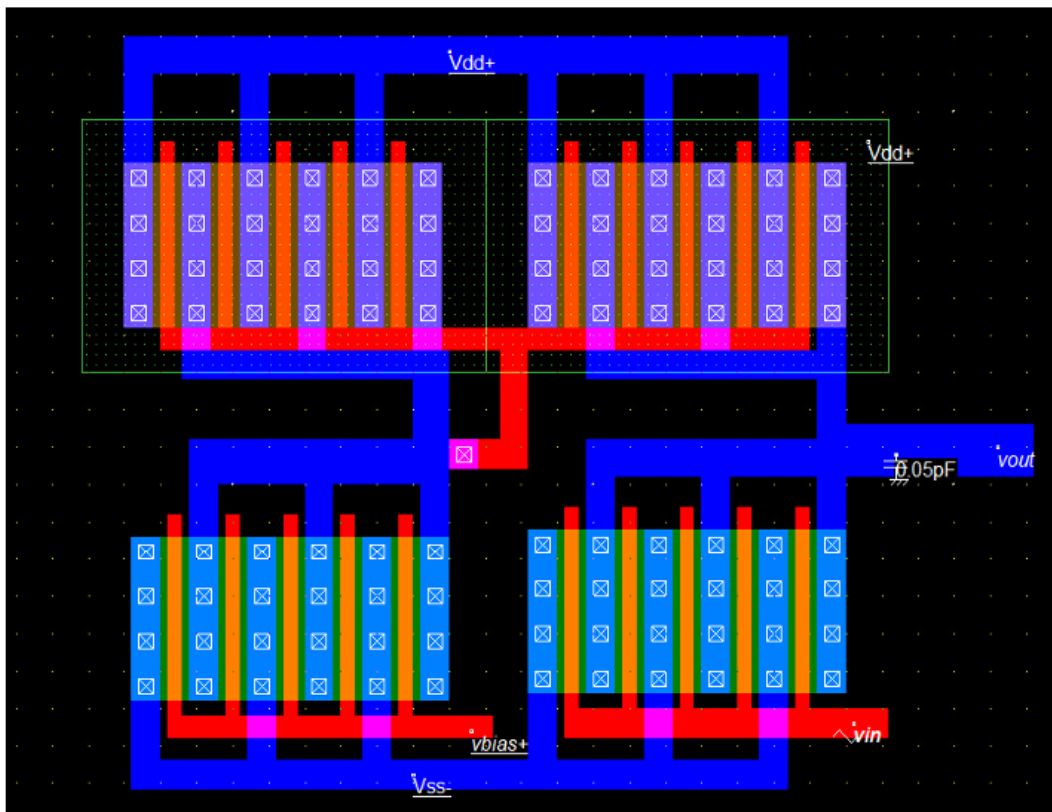
..... (GHz)

i) Transient Analysis waveform:**AC Response:****LAYOUT DIAGRAM:**

PMOS: $W=1000\text{nm}$, $L=40\text{nm}$

NMOS: $W=1000\text{nm}$, $L=40\text{nm}$

Number of Fingers: 5



SPECIFICATIONS:

i) Add a Sinus

Label name: V_{in}

Amplitude (V): 0.1

Frequency (MHz): 500.000

Offset (V): 0.542

Increase frequency: Case i): 0.000

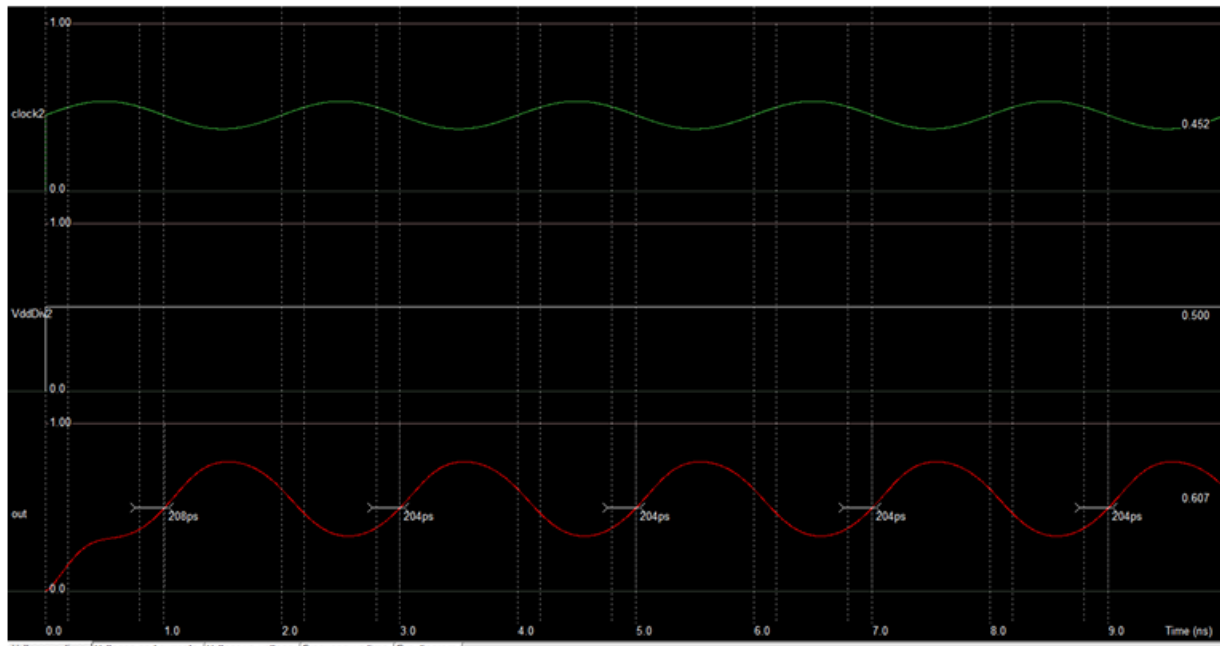
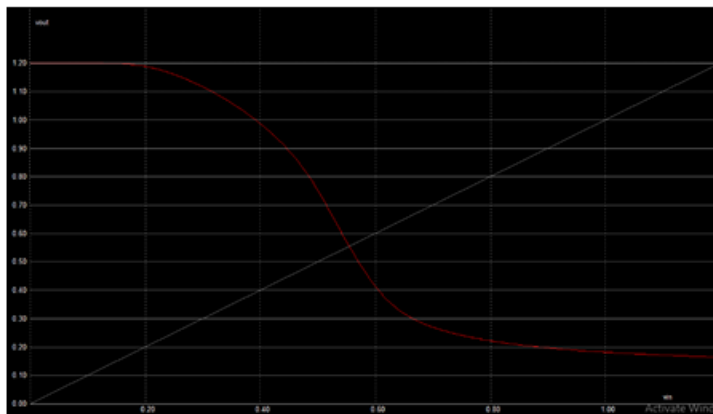
Case ii): 2

ii) Add VDD supply

label name: V_{bias}

Dc voltage level (V): 0.600

Select $VDD^2/2 = 0.600V$

Layout output waveform:**Voltage versus voltage output waveform:**

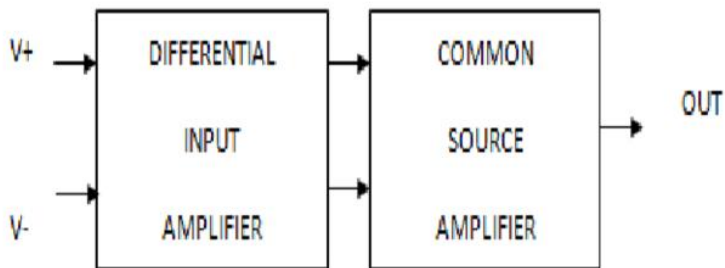
RESULTS: The schematic and layout for the common source amplifier is drawn and verified.

The gain of the common source amplifier is.....

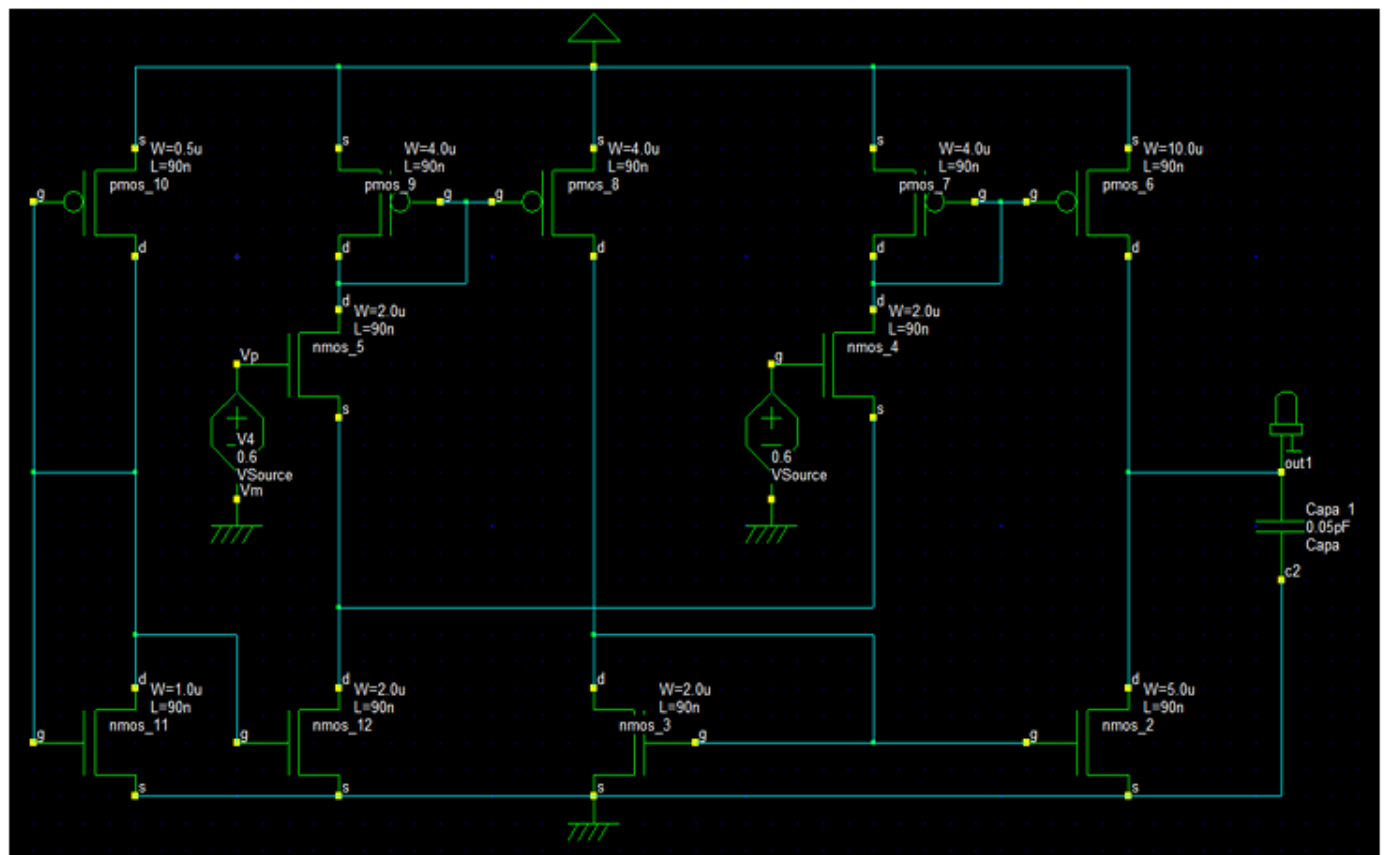
10. OPERATIONAL AMPLIFIER

AIM: To simulate the schematic of the two stage Operational Amplifier.

OBSERVATION:



SCHEMATIC DIAGRAM:



SPECIFICATIONS:**i) Voltage source:** Simple**DC Parameters:**

Values (V): 0.6

ii) Voltage source:**DC Parameters:**

Values (V): 0.6

Sinus parameters: V_O (V) =0.6 V_a (V) =0.05 T_D (ns) =0.0 Θ =0.0**Case i) frequency (MHz) =400****Case ii) frequency (MHz) =4000****Case iii) frequency (MHz) =10000****Load Capacitance:** 0.05 pF**Output waveform Observation****i) Transient Response:****Analysis:** Transient**Frequency/Time:**

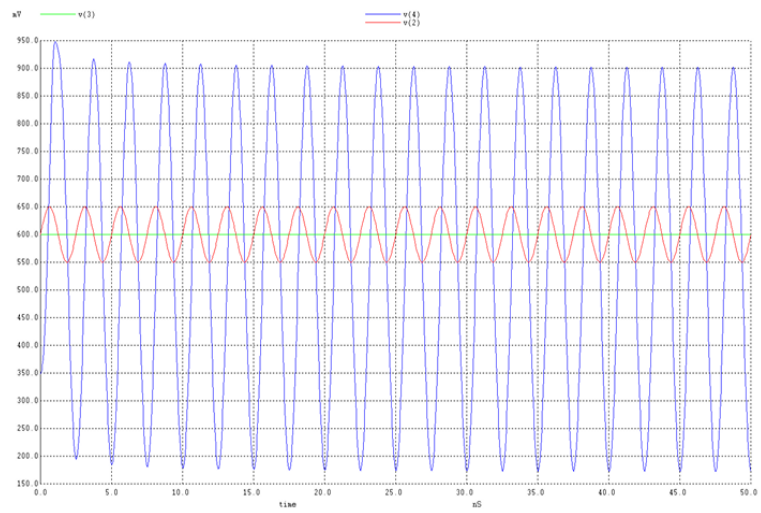
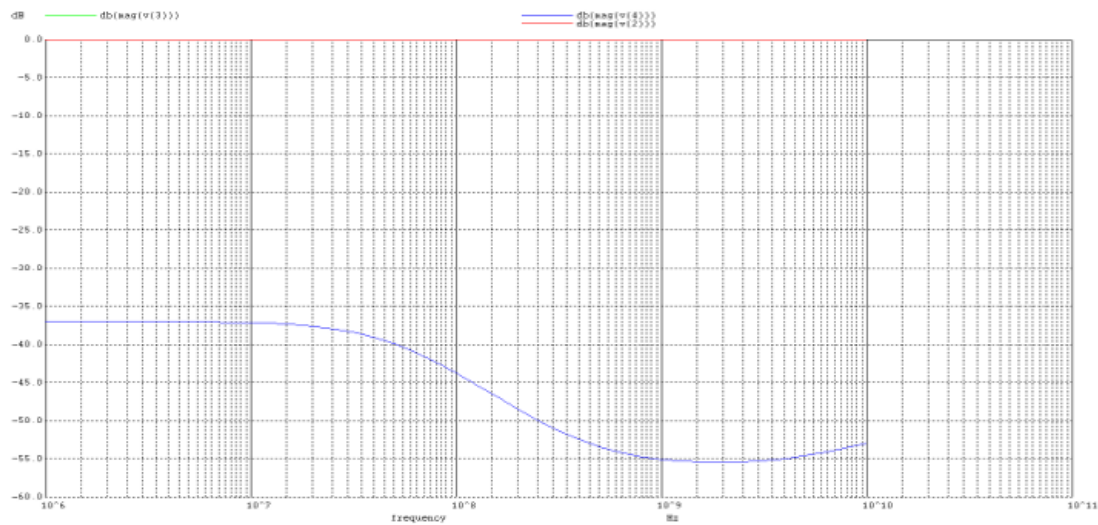
From 1ns to 100ns

Plot: Linear**ii) AC Response:****Analysis:** AC**Frequency/Time:**

From 1MHz to 5GHz

Plot: dB**Gain Calculation****i) Transient Analysis:****Input:****Output:****Gain:** $\frac{\text{Output}}{\text{Input}} =$ **ii) AC Analysis:**

..... (GHz)

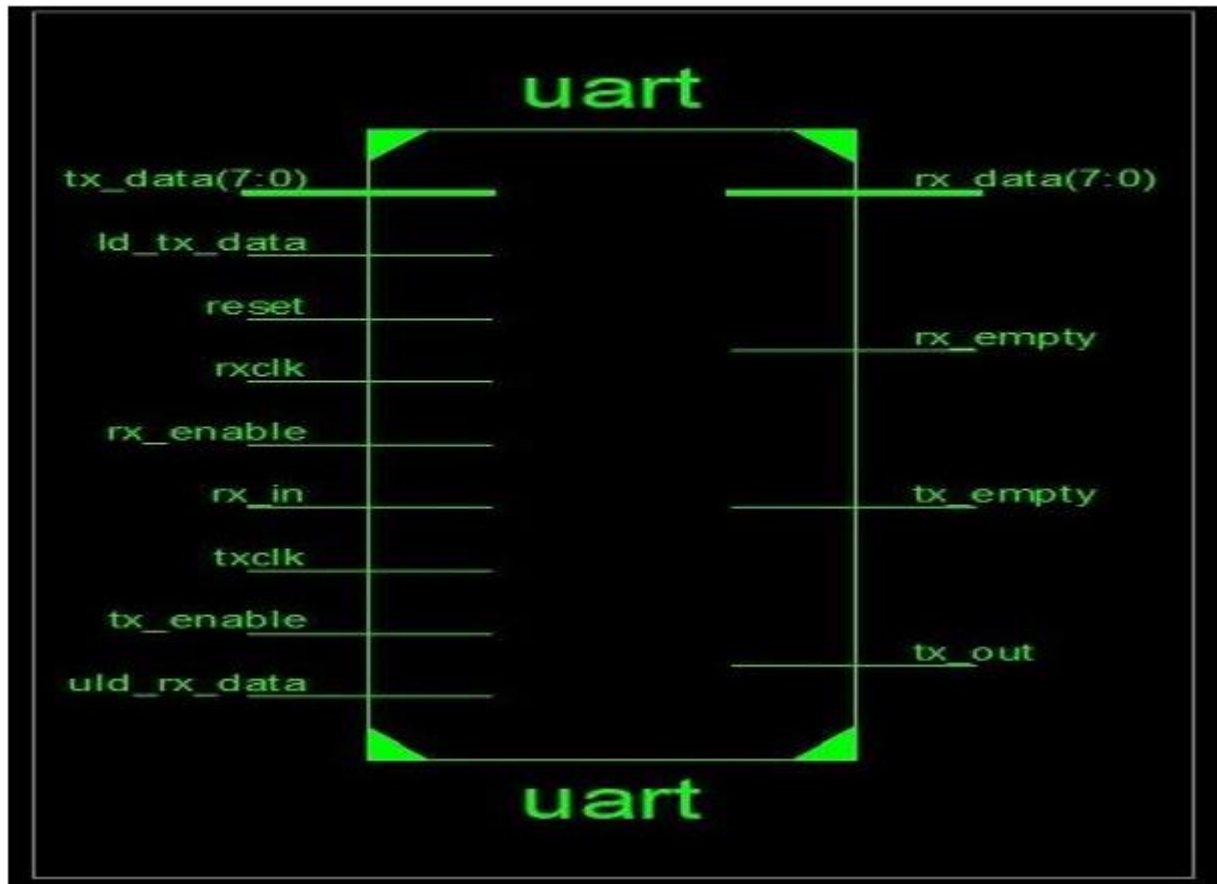
Transient Analysis Waveform:**AC Analysis Waveform:**

RESULTS: The schematic for the operational amplifier is drawn and verified the following transient analysis and AC analysis.

11.UART

AIM: To write Verilog code for UART and its test bench for verification and observe the waveform and synthesize the code.

Block Diagram:



PROGRAM:

```
module uart (
    reset      ,
    txclk      ,
    ld_tx_data ,
    tx_data    ,
    tx_enable  ,
```



```
tx_out      ,
tx_empty    ,
rxclk       ,
uld_rx_data ,
rx_data     ,
rx_enable   ,
rx_in       ,
rx_empty

);

// Port declarations

input  reset      ;
input  txclk      ;
input  ld_tx_data ;
input [7:0] tx_data ;
input  tx_enable  ;
output tx_out     ;
output tx_empty   ;
input  rxclk      ;
input  uld_rx_data ;
output [7:0] rx_data ;
input  rx_enable  ;
input  rx_in      ;
output rx_empty   ;
reg [7:0] tx_reg   ;
reg     tx_empty   ;
```

```
reg      tx_over_run  ;
reg [3:0] tx_cnt      ;
reg      tx_out       ;
reg [7:0] rx_reg      ;
reg [7:0] rx_data     ;
reg [3:0] rx_sample_cnt ;
reg [3:0] rx_cnt      ;
reg      rx_frame_err ;
reg      rx_over_run  ;
reg      rx_empty     ;
reg      rx_d1        ;
reg      rx_d2        ;
reg      rx_busy      ;

// UART RX Logic

always @ (posedge rxclk or posedge reset)

if (reset) begin

    rx_reg      <= 0;
    rx_data     <= 0;
    rx_sample_cnt <= 0;
    rx_cnt      <= 0;
    rx_frame_err <= 0;
    rx_over_run  <= 0;
    rx_empty     <= 1;
    rx_d1        <= 1;
    rx_d2        <= 1;
```

```
rx_busy    <= 0;
end else begin
    // Synchronize the asynch signal
    rx_d1 <= rx_in;
    rx_d2 <= rx_d1;
    // Uload the rx data
    if (uld_rx_data) begin
        rx_data <= rx_reg;
        rx_empty <= 1;
    end
    // Receive data only when rx is enabled
    if (rx_enable) begin
        // Check if just received start of frame
        if (!rx_busy && !rx_d2) begin
            rx_busy    <= 1;
            rx_sample_cnt <= 1;
            rx_cnt      <= 0;
        end
        // Start of frame detected, Proceed with rest of data
        if (rx_busy) begin
            rx_sample_cnt <= rx_sample_cnt + 1;
            // Logic to sample at middle of data
            if (rx_sample_cnt == 7) begin
                if ((rx_d2 == 1) && (rx_cnt == 0)) begin
                    rx_busy <= 0;
```

```
end else begin

    rx_cnt <= rx_cnt + 1;

    // Start storing the rx data

    if (rx_cnt > 0 && rx_cnt < 9) begin

        rx_reg[rx_cnt - 1] <= rx_d2;

    end

    if (rx_cnt == 9) begin

        rx_busy <= 0;

        // Check if End of frame received correctly

        if (rx_d2 == 0) begin

            rx_frame_err <= 1;

        end else begin

            rx_empty    <= 0;

            rx_frame_err <= 0;

            // Check if last rx data was not unloaded,

            rx_over_run <= (rx_empty) ? 0 : 1;

        end

    end

end

end

end

end

end

if (!rx_enable) begin

    rx_busy <= 0;

end
```

```
end

// UART TX Logic

always @ (posedge txclk or posedge reset)

if (reset) begin

    tx_reg    <= 0;

    tx_empty   <= 1;

    tx_over_run <= 0;

    tx_out     <= 1;

    tx_cnt     <= 0;

end else begin

    if (ld_tx_data) begin

        if (!tx_empty) begin

            tx_over_run <= 0;

        end else begin

            tx_reg <= tx_data;

            tx_empty <= 0;

        end

    end

end

if (tx_enable && !tx_empty) begin

    tx_cnt <= tx_cnt + 1;

    if (tx_cnt == 0) begin

        tx_out <= 0;

    end

    if (tx_cnt > 0 && tx_cnt < 9) begin

        tx_out <= tx_reg[tx_cnt -1];

    end

end
```

```
end

if (tx_cnt == 9) begin

    tx_out <= 1;

    tx_cnt <= 0;

    tx_empty <= 1;

end

end

if (!tx_enable) begin

    tx_cnt <= 0;

end

end

endmodule
```

Testbench:

```
module UART_testbench();

reg txclk, reset,tx_enable,ld_tx_data,rxclk,rx_enable;

wire tx_out,tx_empty,rx_in,rx_empty;

reg[7:0] tx_data;

wire[7:0] rx_data;

wire uld_rx_data;

uart U1 (

    .reset(reset),

    .txclk(txclk),

    .ld_tx_data(ld_tx_data),

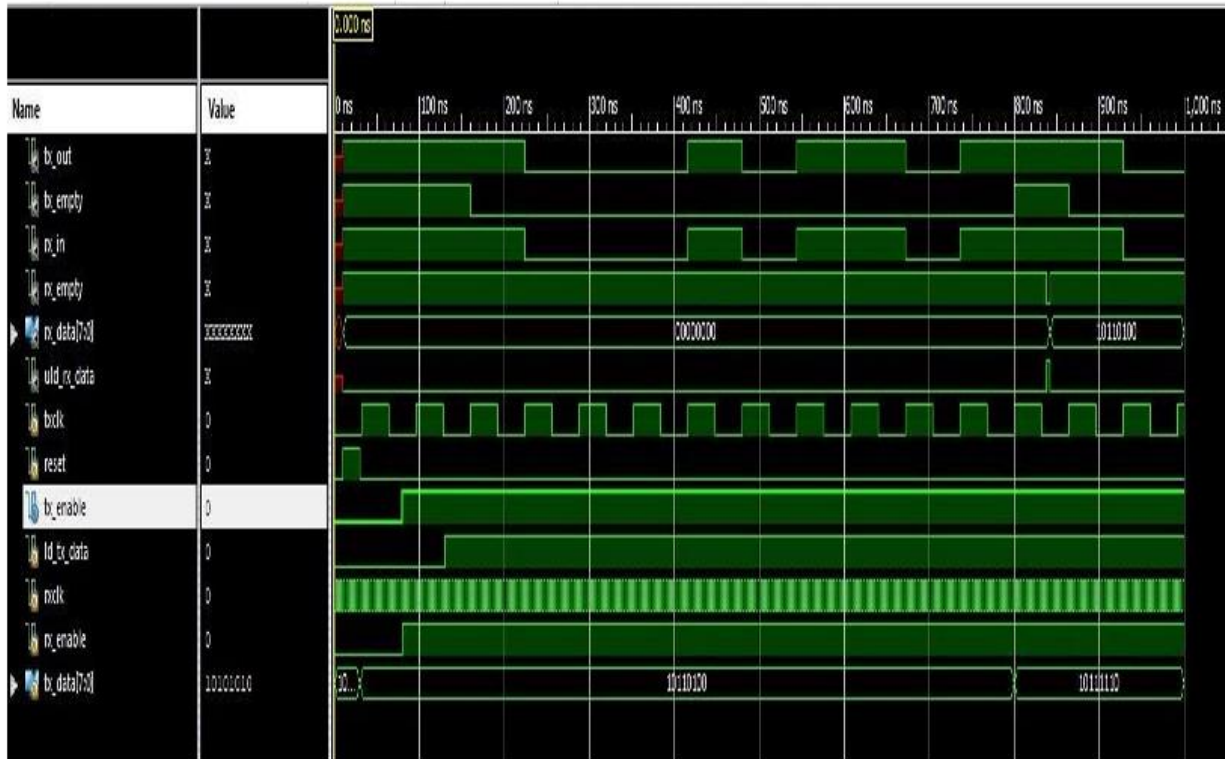
    .tx_data(tx_data),
```

```
.tx_enable(tx_enable),
.tx_out(tx_out),
.tx_empty(tx_empty),
.rxclk(rxclk),
.uld_rx_data(uld_rx_data),
.rx_data(rx_data),
.rx_enable(rx_enable),
.rx_in(rx_in),
.rx_empty(rx_empty)
);
assign rx_in=tx_out;
assign uld_rx_data=~rx_empty;
always@(reset,tx_empty)
begin
if(reset) begin
tx_data=170;
end
else if (tx_empty) begin
tx_data=tx_data+10;
end
end
initial begin
txclk=0;
forever #32 txclk=~txclk;
end
```

```
initial begin
    rxclk=0;
    forever #2 rxclk=~rxclk;
end

initial begin
    reset=0;
    ld_tx_data=0;
    tx_enable=0;
    rx_enable=0;
    tx_data=170;
    #10;
    reset=1;
    #20;
    reset=0;
    #50;
    tx_enable=1;
    rx_enable=1;
    #50;
    ld_tx_data=1;
    #400;
end

endmodule
```

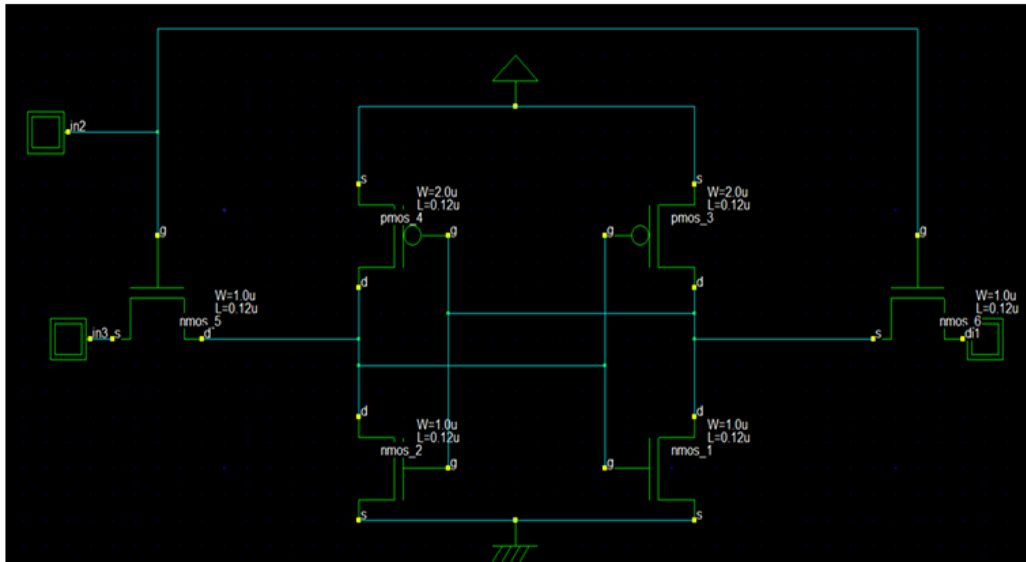

WAVE FORM:

RESULT: Verilog code for UART and its test-bench for verification are written, the waveform is observed and the code is synthesized.

12.Implementation of 6T-SRAM

AIM: To simulate the Schematic Capture of 1-bit 6T -SRAM and Creating Layout of Bit Cell Using microwind 3.8

SCHEMATIC DIAGRAM:



SPECIFICATIONS:

PMOS: $W_p=2.0u$, $L=0.12u$

NMOS: $W_n=1.0u$, $L=0.12u$

WAVEFORM FOR WRITE AND READ OPERATIONS:



Fig:Read Operations

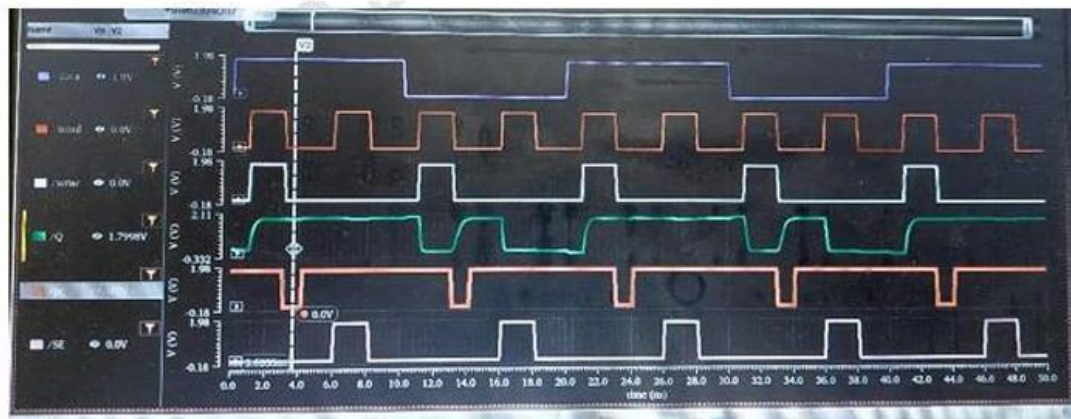


Fig: Write Operations

RESULTS: The schematic of 1bit 6T SRAM is drawn and verified.

Viva Question

1. Why don't we use just one NMOS or PMOS transistor as a transmission gate?
2. What are set up time & hold time constraints? What do they signify?
3. Explain Clock Skew?
4. Why is NAND gate preferred over NOR gate for fabrication?
5. What is Body Effect?
6. Why is the substrate in NMOS connected to Ground and in PMOS to VDD?
7. What is the fundamental difference between a MOSFET and BJT ?
8. Why PMOS and NMOS are sized equally in a Transmission Gates?
9. What happens when the PMOS and NMOS are interchanged with one another in an inverter?
10. Why are pMOS transistor networks generally used to produce high signals, while nMOS networks are used to produce low signals?
11. What is Latch Up? Explain Latch Up with cross section of a CMOS Inverter. How do you avoid Latch Up?
12. Difference between Synchronous and Asynchronous reset.
13. What is DRC ?
14. What is LVS ?
15. What is RCX ?
16. What are the differences between SIMULATION and SYNTHESIS?
17. What is a counter?
18. What are the differences between flipflop and latch?
19. How can you convert JK flipflop into Jk?
20. What are different types of adders?
20. Give the excitation table for JK flipflop?
21. Give the excitation table for SR flipflop?

22. Give the excitation table for D flipflop?
23. Give the excitation table for T flipflop?
24. What is the race around condition?
25. What is an amplifier?
26. What is an op-amp?
27. What is differential amplifier?
28. What is elaboration?
29. What is transient analysis?
30. What is DC analysis?
31. What is AC analysis?